

Chapter 4

Chapter 4: Instruction Set, Addressing Modes, Directives & Tools

- 4.1 Data Formats
- 4.2 Labelling rules
- 4.3 Assembler Directives
- 4.4 Addressing Modes
- 4.5 PIC 18 Instruction Set
 - 4.5.1 Data Transfer
 - 4.5.2 Arithmetic
 - 4.5.3 Logical
 - 4.5.4 Bitwise
 - 4.5.5 Branch Transfer
 - 4.5.6 Table/ROM Program Memory
- 4.6 Overview of MPLAB Simulator

4.1 Data Formats

A microcontroller performs operations on hexadecimal numbers and produces a result that generally is also a hexadecimal number. If a decimal result is desired the result is then converted to a decimal value using a different instruction. Similarly, a user may choose to enter the input data in any choice of data formats while programming and entering the data in the Microchip's MPLAB IDE - Integrated Development Environment.

The common data formats are Hexadecimal, Decimal, Binary and ASCII – American Standard Code for Information Interchange. The different formats representation used are presented below. Also note that these formats may vary from assembler to assembler.

Hexadecimal Representation

The hexadecimal numbers are represented using the following.

Example: 50, h'9C', H'8A', 8000h, 9000H, 0x2000, 0X4000.

- A number when succeeded by a h or H as 800H or 9000h
- A number entered as just 50 is considered as 50H. In assemblers of 8085, 8086 and 8051 if nothing is appended at the front or to the back it is considered as a decimal value of 50d.
- A number in single quotes if preceded by h or H as in h'9C' is considered 9CH.
- A number may be preceded by a 0x as in 0x2000 or 0X3000
- If a hexadecimal number has A-F as its higher digit, it needs to be preceded by a 0 as in 0E7H.

Decimal Representation

The decimal numbers are represented using the following.

Example d'56', D'45', .12

- A number in single quotes if preceded by D or d as in d'98' or D'98' is considered 98.
- A number if preceded by a . (Dot) i.e (dot value) as in .98 is considered 98

Binary Representation

The binary numbers are represented using the following.

Example b'1010', B'0101'

- A number in single quotes if preceded by b or B as in b'1011' or B'1011' is considered binary 1011.

ASCII Representation

The ASCII numbers are represented using the following.

Example a'4', A'8', '7'

- A number in single quotes if preceded by a or A as in a'5' or A'5' is considered as 35H which is ASCII equivalent for number 5.

- A number in single quotes as in '6' is considered as 36H which is ASCII equivalent for number 6.
- For more than one character, assembler directive DB - Define Byte need to be used.

4.2 Labelling rules

Any program whether it is a high-level language or an assembly language is bound to use identifiers. Identifiers are the names given to a particular data value, address, variable, function or macro etc.

As part of good and standard programming practices each identifier is named such that a program is easy to read, understand, document and maintain. The identifiers are governed by labelling rules in an IDE and may vary from one IDE to other. General rules are as follows.

- Each identifier/label name needs to be unique.
- It makes use of
 - Alphabetic letters in any case(upper/lower)
 - digits 0 to 9
 - Special characters

▪ Question Mark	'?'
▪ Dot or Period	'.'
▪ At	'@'
▪ underline	'_'
▪ dollar sign	'\$'
- First character of label should be an alphabetic character.
- Should not use reserved identifiers like mnemonics of instructions – MOVWL, ADDWL. Other additional identifiers may be used which vary from different IDE's.

4.3 Assembler Directives

An Assembler Directive as the name suggest is a directive or command to the assembler. They are used direct the assembler in performing certain tasks as part of assembly program conversion or to control the programming process. They are only directives to the assembler, part of the source code in editor but are not part of the code that is loaded into the memory of a microcontroller for execution. Whereas an instruction is a command used to make the microcontroller perform an operation in the CPU and is stored in the memory of microcontroller.

Assembler Directives help the programmer in reducing and optimizing the written code. Since some directives are generally replaced with an equivalent instruction, Assembler

directives are also called as Pseudo Instructions. Equivalent Compiler directives are present for High Level Language programming.

MPASM Assembler that is part of MPLABX IDE has the following types of directives.

- Control Directives
- Conditional Assembly Directives
- Data Directives
- Listing Directives
- Macro Directives
- Object File Directives

Some of the commonly used directives are presented in detail here. Interested readers may refer to the MPLABX IDE documentation for the other directives along with additional details. **The Assembler directives presented here is a compiled gist from the MPLABX IDE Help Documentation.**

Also note that PIC18 stores code in program memory in a two-byte (1 Word) format. i.e each location consists of 16-bit.

Control Directives

These directives control how the code is assembled.

- ORG → Set Program Origin
- END → End Program Block
- #INCLUDE → Include Additional Source File
- RADIX → Specify Default Radix
- EQU → Define an Assembler Constant
- SET → Define an Assembler Variable
- CONSTANT → Declare Symbol Constant
- VARIABLE → Declare Symbol Variable
- #DEFINE → Define a Text Substitution Label
- #UNDEFINE → Delete a Substitution Label
- PROCESSOR → Set Processor Type

ORG - Set Program Origin

The ORG directive is used to set the origin or the starting address from where the program code or subsequent instructions is stored. This directive is generally used in ASM-Assembly program files where code is to be placed at a specific location.

Syntax: ORG Expression Ex: ORG 0x30

ORG 0x30 indicates that the following instructions are stored from 0x30 address location. In PIC18 only even numbered expression is allowed as LSB of PC is neglected.

END - End Program Block

The END directive is used to indicate end of the program to the assembler. There should be at least one END directive in any Assembly program and only one in a single assembly file program. Any instructions or directives after the end are discarded or not considered by the assembler.

Syntax: END

#INCLUDE - Include Additional Source File

The #include directive is used to make use of statements that are present in a different file. #include directive is used at least once to include the standard header file related to the selected microcontroller that contains predefined register, bit, names, other symbols and constants etc so that the user/programmer need not define them separately in their code.

Syntax

#include include_file	EX:	#include f1.h
#include "include_file"		#include "f1.h"
#include <include_file>		#include <f1.h>

When an include is performed, the file specified is read as source code and is equivalent to inserting/replacing the 'include file' statement with all the contents of the specified file. A total of up to 5 nesting levels are permitted and a total of up to 255 include files. If the file name consists of spaces, it needs to be enclosed in quotes or angle brackets. A specific path may be assigned for a specific directed search. The default search order is i) current working directory ii) source file directory iii) MPASM assembler executable directory.

RADIX - Specify Default Radix

The RADIX directive is used for setting the default (data format) radix for data expressions. The default data format (Radix) is HEX. The possible values are hex for hexadecimal (base 16), dec for decimal (base 10), oct for octal (base 8). The LIST directive may also be used to specify a radix. Care needs to be taken since after changing the radix all the following values follow the new radix format.

Syntax

radix default_radix	Ex: radix dec	or	radix hex etc
---------------------	---------------	----	---------------

The list directive option r= may be used to change the radix in the user's code.

EQU – Define an Assembler Constant

The EQU directive is used to assign the value of expr to label. It is generally used for assigning address location in RAM to a variable name. A name once assigned with a value using an EQU directive cannot be reassigned with a different value. This directive is not to be used in building a linked project. The RES directive inside a data section directive (idata, udata) may be used instead.

Syntax

label equ expr

Ex: My_Loc = 0x23

SET – Define an Assembler Variable

The SET directive is used to assign the value of expression to a label. It is functionally equivalent to the EQU directive with the exception that assigned/set values may subsequently be changed/alterd by a different set directive. Symbols declared as EQU cannot be reset or changed whereas symbols declared as variable using SET directive may be changed.

Syntax

label set expr

Ex: My_Loc set 0x23

CONSTANT – Declare Symbol Constant

The CONSTANT directive is used to create symbols for use in MPASM assembler expressions. The expression must be completely resolvable during assignment. Once initialized the constants may not be reset. This is similar to declaring constants in C and is also similar to EQU directive. List of named sequential symbols may be assigned using CBLOCK directive. Symbols declared as constant cannot be reset or changed whereas symbols declared as variable using SET directive may be changed.

Syntax

constant label = expr [...],label=expr]

Ex: constant My_Loc = 0x23

VARIABLE – Declare Symbol Variable

The VARIABLE directive is used to create symbols for use in MPASM assembler expressions. It is functionally equivalent to SET directive. The symbol names declared using VARIABLE directive need not be initialized while declaration. It is commonly used for conditional assembly code. Reading of a variable and writing of new value to that variable value cannot be performed in same instruction operand and require step by step assignments in separate lines/instructions.

Syntax

variable label[=expr][,label[=expr]...]

Ex: variable P1, P2=0

#DEFINE – Define a Text Substitution Label

The #DEFINE directive is used to define a values, constants, text substitution string. A defined label name that is defined and when used in a program, wherever the name is encountered the equivalent defined string will be substituted. This directive is equivalent to #define of ANSI 'C' standard.

Syntax

#define name [string]

Ex: #define My_Loc 0x23

#UNDEFINE – Delete a Substitution Label

The #UNDEFINE directive is used to remove the previously defined symbol (using the #define directive) from the symbol table. A symbol name initialized/defined with a value earlier using #define need to be undefined using #UNDEFINE directive before it is reinitialized/redefined.

Syntax

```
#undefine label
```

PROCESSOR – Set Processor Type

The PROCESSOR directive used to set/select the type of processor. Ex: PIC18F458. This directive is generally not required in MPLAB X IDE projects as the processor is selected in the process of creation of a project. If required, it need to be used in code using the PROCESSOR or LIST directive option p= to set the processor.

Syntax

```
processor processor_type
```

Conditional Assembly Directives

The conditional assembly directives are used to assemble certain sections of code that satisfactorily meets a given condition. It is used to control and select the code that need to be assembled but not how the code is executed.

- IF → Begin Conditionally Assembled Code Block
- ELSE → Begin Alternative Assembly Block to IF Conditional
- ENDIF → End Conditional Assembly Block
- IFDEF → Execute If Symbol Has Been Defined
- IFNDEF → Execute If Symbol Has Not Been Defined
- WHILE → Perform Loop While Condition is True
- ENDW → End a While Loop

A simple example for the 'If' - Begin Conditionally Assembled Code Block is as follows.

```
variable Var1      ; Variable used to define required configuration of PORTA & PORTB
Var1 set 0         ;Set config to 0
if Var1==0x0       ;If config==0x0 is true, ;assemble the mnemonics up to
    MOVLW 0x00      ;the directive 'else'. Set up PORTA as output.
    MOVWF TRISA
    MOVLW 0xFF
    MOVWF TRISB

else
    MOVLW 0x00      ;If config==0x0 is false, ;assemble the mnemonics up to
```



```

MOVWF TRISB      ;the directive 'endif'. Set up PORTB as output.
MOVLW 0xFF
MOVWF TRISA
endif

```

You may refer to MPLAB X IDE documentation for the details of other Conditional Assembly Directives.

Data Directives

The Data directives control the allocation of memory and provide a way to refer to data items symbolically, i.e., by meaningful names.

- DATA → Create Numeric and Text Data
- DB → Declare Data of One Byte
- DW → Declare Data of One Word
- DE → Declare EEPROM Data Byte
- FILL → Specify Program Memory Fill Value
- RES → Reserve Memory
- CBLOCK → Define a Block of Constants
- ENDC → End an Automatic Constant Block
- CONFIG → Set Processor Configuration Bits
- __CONFIG → Set Processor Configuration Bits
- __BADRAM → Identify Unimplemented RAM
- __BADROM → Identify Unimplemented ROM
- __MAXRAM → Define Maximum RAM Location
- __MAXROM → Define Maximum ROM Location
- __IDLOCS → Set Processor ID Locations

DATA – Create Numeric and Text Data

The DATA directive is used to initialize one or more words of program memory with data in the form of constants, relocatable or external labels, or expressions, ASCII character strings, text_string that is enclosed in single quotes for one character or double quotes for strings. The PIC18 family devices store the first character in the least significant byte of the word which is contrary to all other PIC families.

If the data is a single character item, it is placed into low byte of the word, and if the data is a string, the characters are packed and stored in two to form a word. For an odd number of string characters, the final high byte is set to store a zero.

Syntax

[label] data expr[,expr,...,expr][label] data "text_string"["text_string",...]

- i. Example: MY_St1 data "P","I","C"

This is stored as 3 program memory location words in PIC18. The 3 locations are filled with ASCII; equivalent of "P", "I" and "C". The data produces three 16-bit numbers: 0050h, 0049h, and 0043h. 50h, 49h and 43h are ASCII equivalents of "P", "I" and "C" respectively

- ii. Example: My_St2 data "MCU"

This is stored as 2 program memory location words in PIC18. The 2 locations are filled with two words (16-bit numbers), each representing ASCII equivalent of two characters. The high byte of second location is assigned a value of 0 or the NULL character. The two-word data that is stored is 4950h and 0043h. 4950h represents "I" and "P".

- iii. Example: My_St3 data 0x1234,0x5678

This is stored as 2 program memory location words. The 2 locations are filled with 1234 and 5678 in two consecutive program memory locations.

DB – Define/Data Byte

The DB directive is used to declare data of one byte. It reserves program memory words with 8-bit values. In case of multiple expressions, each expression is continuously filled in consecutive bytes until the end of all expressions. For odd number of expressions, the final high byte location is generally assigned a value of 0 or the NULL character, except for cases like code_pack section.

Syntax

[label] db expr[,expr,...,expr]

- i. Example: My_St1 DB "P","I","C" is stored similar to example 1 in three different word locations as in DATA directive.
- ii. Example: My_St2 DB 0x12, 0x34 are stored as 0x3412 in one program memory location.
- iii. Example: My_St1 DB "PIC" is stored similar to example 2 in two different word locations as in DATA directive.

DW – Define/Data Word

The DW directive is used to initialize one or more words(16-Bit). It reserves program memory words with 16-bit values for data, initializing that space to specific values. The DW directive functions similar to db. The expressions may be literal strings and the whole word value(s) are stored in consecutive memory locations, with increment of location counter for every word.

Syntax

[label] dw expr[,expr,...,expr]

- i. Example: My_St1 DW "P","I","C" is stored similar to example 1 in three different word locations as in DATA directive.
- ii. Example: My_St2 DW 0x39, 0x45, 0x1234 are stored as 0X0039 in one program memory location and 0x0045, 0x1234 in others.
- iii. Example: My_St1 DB "PIC" is stored similar to example 2 in two different word locations as in DATA directive.

DE – Declare EEPROM Data Byte

The DE directive is used for initializing data in the EEPROM data memory region with EEPROM data FLASH. The reserve memory word bytes are packed. For odd number of bytes, the final high byte location of word is generally assigned a value of 0 or the NULL character, except for cases like code_pack section. This directive can be used at any location for any processor. The start of EEPROM data memory need to be specified in user's code as shown in the example below. The EEPROM start address for PIC18 is 0xF00000.

Syntax

[label] de expr [, expr, ..., expr]

Example:

```
org 0xF00000          ;The absolute address F00000h is mapped to the 0000
                      ;location of EE data memory in PIC18 devices. A Data pr
                      ;character table can be created from any address in
                      ;EEPROM data memory.

My_Loc DE "PIC18F"    ;6 EEPROM data memory locations (from 0) will be
                      ;filled with 6 ASCII characters.
```

FILL – Specify Program Memory Fill Value

The FILL directive is used to force known data/instructions into unused program memory (fill data in memory location). It is used to specify successive program memory locations with a constant or an assembly instruction like goto or branch instructions if bounded by parentheses. Such instructions make sure that if branch occurs in code to an unused area at run-time, a fail-safe condition occurs. It generates count (byte count) occurrences of the program word or byte, expr. Since PIC18 program memory is word based, the count of locations to be filled has to be even.

Syntax

[label] fill expr, count

Example:

```
fill (nop), 8          ;Fills 4 locations (8 bytes) with NOP
fill (goto start),6    ;Fills 6 locations (each location is 2 bytes & each goto start is 2
                      ;words, so 3 instruction words each with 2 word wide instruction
                      ;<goto start>
```

fill 801A, 4 ;Fills 2 locations (4 bytes) with 801Ah

Note: An example for expression can be START - \$; where START is a label. \$ indicates current address. In this case the count value equals the number of bytes between label start and current address \$. GOTO \$ is equivalent to HERE: GOTO HERE

RES – Reserve Memory (bytes)

The RES directive is used to reserve a specified number of bytes. The memory location pointer is incremented from its current location by the value specified in mem_units.

Syntax

[label] res mem_units

Example: Length res 2 ; Reserves two locations of memory for the label Length.

Linker allocates addresses of the memory locations.

CBLOCK – Define a Block of Constants & ENDC – END a Constant Block

The CBLOCK directive is used to define a list of named sequential symbols constants in data memory. This helps in assigning address offsets to many labels till an ENDC directive is encountered. The ENDC directive is used to terminate the cblock or indicate the end of the cblock. The starting address value for first label name is specified as part of expression. If no expression is present, the starting address is 0 if there is no earlier cblock, and if there is an earlier cblock the address value is assigned a value higher than previous label. Multiple names in a line need to be separated by commas.

Syntax

```
cblock [expr]
    label[:increment][,label[:increment]]
endc
```

Example1 :

```
cblock 0x30
    P1, P2 ; P1 will be assigned 30, P2 with a 31 and so on
    P3, P4 ; P4 is assigned 33.
endc
```

Example 2:

```
CBLOCK 0x30 ;Define a block of variables starting at 30H in data memory.
    Var1:2 ;The size of label Var1 is 2 bytes, assigned address 30H & 31H
    Var2 ; Address 32H is assigned to the label Var2.
    Var3 ;Address 33H is assigned to the label Var3.
ENDC
```

CONFIG – Set Processor Configuration Bits

The CONFIG directive is used to define a list of Configuration bit setting definitions. This is used to set or reset the one or more configuration bits of PIC18 for its operation. The details of the Configuration bits are found in the included (*.inc) files for the respective processor or in the PIC18 Configuration Settings Addendum (DS51537). Multiple settings may be defined on. Configuration bit settings can be done in a single line (need to be separated by commas) or in separate lines. __config directive can also be used similar to CONFIG, but this is not recommended for new code and both should not be used in the same code. This directive is placed in source code so as to ensure the correct (pre-set to desired values) configuration settings of the device are loaded into it when the code is compiled/assembled into a hex file and programmed.

Syntax

```
config setting=value [, setting=value]
```

Note:

Before this directive is used, a PIC18 MCU must be declared through the command line, the list directive, the processor directive, a project if using MPLAB X IDE or Configure>Select Device if using MPLAB IDE. Place configuration bit assignments at the beginning of your code. Use the configuration options (setting=value pairs) listed in the standard include (*.inc) file or the addendum.

Care need to be taken so as not to assign values multiple times for the same configuration bit as an error is generated if the same bit is assigned a value more than once, i.e.,

- CONFIG CP0=OFF, WDT=ON
- CONFIG CP0=ON ;(An error will be issued since CP0 is assigned twice)

Example:

- CONFIG CP0=OFF ;code protect disabled
- CONFIG OSCS=ON, OSC=LP ;Oscillator switch enabled, RC oscillator with OSC2 as I/O pin.
- CONFIG BOR=ON, BORV=25 ;Brown-OutReset enabled, BOR Voltage is 2.5v
- CONFIG WDT=ON, WDTPS=128 ;Watch Dog Timer enable, Watch Dog Timer PostScaler count - 1:128
- CONFIG CCP2MUX=ON ;CCP2 pin Mux enabled
- CONFIG STVR=ON ;Stack over/underflow Reset enabled

Listing Directives

Listing directives control the MPASM assembler listing file format. These directives allow the specification of titles, pagination, and other listing control. Some listing directives also control how code is assembled.

- LIST → Listing Options
- NOLIST → Turn off Listing Output
- MESSG → Create User Defined Message
- ERROR → Issue an Error Message
- ERRORLEVEL → Set Message Level
- TITLE → Specify Program Title
- SUBTITLE → Specify Program Subtitle
- SPACE → Insert Blank Listing Lines
- PAGE → Insert Listing Page Eject

LIST – Listing Options

The LIST directive is used to control the assembly process or formatting the listing output file. Various options available are presented in the table given below.

Table 4.1: LIST Directive Options

Option	Default	Description
b=nnn	8	Set tab spaces.
c=nnn	132	Set column width.
f= format {INHX32, INHX8M, or INHX8S}	INHX8M	Set the hex file output format. Note: Hex file format is set through Project Properties window in MPLAB X IDE
free	FIXED	Use free-format parser. Provided for backward compatibility.
fixed	FIXED	Use fixed-format parser.
mm={ON OFF}	On	Print memory map in list file.
n=nnn	60	Set lines per page.
p= type	None	Set processor type; Ex: PIC16F877. Processor type is set in MPLAB X IDE through projects creation.
pe= type	None	Set processor type and enable extended instruction set. Ex LIST pe=PIC18F4620 Processor type is set in MPLAB X IDE through projects creation.
r= radix	hex	Set radix: hex, dec, oct. See also radix.
st={ON OFF}	On	Print symbol table in list file.

t={ON OFF}	Off	Truncate lines of listing (otherwise wrap).
w={0 1 2}	0	Set the message level. See also error level.
x={ON OFF}	On	Turn macro expansion on or off.

Note: All list options are evaluated as decimal numbers by default.

Syntax

list [list_option, ..., list_option]

NOLIST – Turn off Listing Output

The NOLIST directive is used to suppresses the information required for the listing file and source-level debugging. This will prevent the ability to debug the source code.

Syntax

nolist

Macro Directives

Macro directives control the execution and data allocation within macro body definitions.

- MACRO → Declare Macro Definition
- ENDM → End a Macro Definition
- EXITM → Exit from a Macro
- LOCAL → Declare Local Macro Variable
- EXPAND → Expand Macro Listing
- NOEXPAND → Turn off Macro Expansion

MACRO – Declare Macro Definition & ENDM

A MACRO directive is used to group a sequence of instructions that can be inserted in the assembly source code by using a single macro call. A Macro must be defined before it is referred in a subsequent code. A macro may refer another macro or itself recursively up to 16 nesting levels. Macros may have optional arguments and need to be less than source line or 200 characters in length.

Syntax

label macro [arg, ..., arg]

The last statement in a MACRO is the ENDM directive that indicates End of a Macro Definition. This terminates a macro definition begun with macro. Every MACRO directive at a certain point should end with an ENDM directive.

Example

```
; Definition of a MACRO
My_Macro1 macro Var1, Var2, Var3
    MOVLW Var1
```

```

MOVWF Var2
ADDWF Var3,0
CALL MY_SUB_ROUTINE1      ; subroutine call
ENDM
; Calling/Using a Macro in main program
My_Macro1 0x01, 0x21, 0x45

```

EXITM – Exit from a Macro

The EXITM directive is similar to ENDM directive and forces immediate return from macro expansion during assembly. This directive is mainly used to prematurely end a macro, or end based a specific condition and is similar to break command in C.

Syntax

```
exitm
```

Example

```

My_Macro1 macro Var1
    if Var1 == 1    ; check for valid file
        exitm
    else
        error "bad file assignment"
    endif
endm

```

LOCAL – Declare Local Macro Variable

The LOCAL directive is used to specify the data elements that are considered to be in local context to the macro. i.e Variables/Labels used only inside a MACRO need to be declared as LOCAL. If a MACRO is called recursively, each invocation will have its local copy. If a macro contains a label Ex: Label for a Branch instruction, local variable, etc they need to be declared as local. Otherwise, an error “Duplicate Label” is generated.

Syntax

```

Preferred:
local label[,label...]

```

Example

```

<main code segment>
:
:
Var1 equ 11    ; global version
Var2 equ 22    ; note that a local variable may now be created and modified.
My_Macro1 macro Var2
    local Var1, L1    ; local len and label

```



```

len set size          ; modify local Var1
L1 res Var1           ; reserve buffer
Var1 set Var1-20
endm                  ; end macro

```

EXPAND – Expand Macro Listing & NOEXPAND – Turn off Macro Expansion

The EXPAND directives is used to expand all macros in the listing file. EXPAND MACRO listing is the default. This directive may be disabled/negated by the occurrence of a subsequent NOEXPAND directive. It turns off macro expansion in the listing file. The EXPAND directive is useful for debugging in small range of code with multiple macros.

Syntax of EXPAND

```
expand
```

Syntax of NOEXPAND

```
noexpand
```

Example:

NOEXPAND	EXPAND
<pre> ADD1 MACRO VAR1, VAR2 MOVLW VAR1 MOVWF POSTINC0 MOVLW VAR2 ADDWF POSTINC1 ENDM : NOEXPAND ;USE MACRO ADD1 ADD1 .50,.16 RESULTING LISTING FILE: 00029 NOEXPAND 00030 ADD1 .50,.16 00031 </pre>	<pre> ADD1 MACRO VAR1,VAR2 MOVLW NUM1 MOVWF POSTINC0 MOVLW VAR2 ADDWF POSTINC1 ENDM : EXPAND ;USE MACRO ADD1 ADD1 .50,.16 RESULTING LISTING FILE: 00029 EXPAND 00030 ADD1 .50,.16 0010 3032 M MOVLW .50 0011 6EEE M MOVWF POSTINC0 0012 3010 M MOVLW .16 0013 26E6 M ADDWF POSTINC1 00031 </pre>

Fig 4.1 Example to demonstrate resulting listing file with & without use of expand listing

Object File Directives

Object file directives are used only when creating an object file.

- GLOBAL → Export a Label
- EXTERN → Declare an Externally Defined Label
- CODE → Begin an Object File Code Section
- CODE_PACK → Begin an Object File Packed Code Section
- ACCESS_OVR → Begin an Object File Overlay Section in Access RAM
- IDATA → Begin an Object File Initialized Data Section
- IDATA_ACS → Begin an Object File Initialized Data Section in Access RAM
- UDATA → Begin an Object File Uninitialized Data Section
- UDATA_ACS → Begin an Object File Access Uninitialized Data Section
- UDATA_OVR → Begin an Object File Overlayed Uninitialized Data Section

GLOBAL – Export a Label & EXTERN – Declare an Externally Defined Label

The GLOBAL directive is used to make the label/symbol names visible to other modules. The EXTERN directive facilitates the use of label/symbol names of the current module but are defined as global in a different module. i.e A label/symbol name declared as global in the current module is made available to other modules when the same label is declared as EXTERN in the module that is using it. Global and EXTERN directives may be used in project containing more than one file. An error will be generated if the global and extern for a label/symbol name are in the same module.

Syntax for global

global label [, label...]

Example

global Var1, Var2

Syntax for extern

extern label [, label...]

Example

extern Function
:
call Function

Main Module	ADD1 Module
GLOBAL Var1 ;The variable 'Var1' declared GLOBAL in Main Module. EXTERN Var2 ;The variable 'Var2' declared EXTERN in Main Module but declared Global in ADD1 module	GLOBAL Var2 ;The variable 'Var2' declared GLOBAL in ADD1 Module. EXTERN Var1 ;The variable 'Var1' declared EXTERN in ADD1 Module but declared Global in Main module.

Fig 4.2 Example that demonstrates use of Global & Extern directive

CODE – Begin an Object File Code Section

The CODE directive is used to declare the beginning of a section of program code. This is similar to having segments in 8086. This helps in organizing the code in different sections. No two sections in a file can have the same name. Code of a section ends automatically and doesn't require end code section. The section ends with the start of another code or data section or an END directive. A label name and a program ROM address may be assigned for a code section. The absolute address may be specified or an address will be assigned at link time if it is not specified. It is used when storing data into program memory (use with db) or the EEPROM data memory (use with de).

Syntax

```
[label] code [ROM_address]
```

Example

```
MyCode_Sec1 code 0x200          ;Absolute address 0x200
    MOLW 0x21
    MOVWF 0x31
    ; Here the code section named MyCode_Sec1 is
    ; placed at program memory 0x200
MyCode_Sec2 code                ;Relocatable address
    MOLW 0x21
    MOVWF 0x31
    ; Here the code section named MyCode_Sec2 is relocated at run time
    ; based on number of locations used in MyCode_Sec1.
```

CODE_PACK – Begin an Object File Code Section

The CODE_PACK directive is similar to CODE directive except that a byte of zero is not appended in high byte data when odd number of bytes are encountered in program code or ROM data. It is used to declare the beginning of an object file packed code section. An example is shown in Fig 4.3.

Syntax

```
[label] code_pack [ROM_address]
```

ACCESS_OVR – Begin an Object File Overlay Section in Access RAM

The ACCESS_OVR directive is used to declare beginning of overlay data section of same name in Access RAM. Specifying the starting address is optional. If specified it is used as the absolute starting address and if no address is specified, it is assigned at link time. This directive allows to reuse access-bank data space. The space is shared by all ACCESS_OVR

sections and program code is not to be placed in this segment. This directive is similar to `udata_acs` & `udata_ovr` directives but declares for Access RAM, uninitialized data.

Syntax

[label] `access_ovr` [RAM_address]

Example

The example in Fig 4.4 here shows P11 overlaid with memory locations used by P1 and P2. P12 is overlaid with memory locations used by P3 and P4. The location 0x01 replaced with 0x11 through first `MOVWF` and location 0x02 replaced with 0x11 through second `MOVWF`.

Example	Example with Address	Result			
P1 code_pack 0x20 DB 1, 2, 3 DB 1, 2, 3	P1 code_pack 0x20 0x20 DB 1, 2, 3 0x23 DB 1, 2, 3	Line	Address	Opcode	DisAssy
		16	001E	FFFF	NOP
		17	0020	0201	MULWF 0x1, ACCESS
		18	0022	0103	MOVLB 0x3
		19	0024	0302	MULWF 0x2, BANKED
		20	0026	0000	NOP
					:
P2 code 0x30 DB 1, 2, 3 DB 1, 2, 3 END	P2 code 0x30 0x30 DB 1, 2, 3 0x34 DB 1, 2, 3 END	25	0030	0201	MULWF 0x1, ACCESS
		26	0032	0003	SLEEP
		27	0034	0201	MULWF 0x1, ACCESS
		28	0036	0003	SLEEP
		29	0038	FFFF	NOP

Fig 4.3 Example that demonstrates use of `code_pack` directive

Example	Main Code	Result					
AC_OV access_ovr P1: res 1 P2: res 1 P3: res 1 P4: res 1 AC_OV access_ovr P11: res 2 P12: res 2	MyC1 CODE MOVLW 0X11 MOVWF P2 MOVWF P12 NOP END	Address	00	01	02	03	04
		000	00	11	11	00	00
		010	00	00	00	00	00
		020	00	00	00	00	00
		030	00	00	00	00	00

Fig 4.4 Example that demonstrates use of `access_ovr` directive

`UDATA_OVR` directive declares the beginning of a section of overlaid uninitialized data. `IDATA` directive declares the beginning of a section of initialized data. `UDATA` directive declares the beginning of a section of uninitialized data. `UDATA_ACS` directive declares the

beginning of a section of access initialized data. IDATA_ACS directive declares the beginning of a section of access uninitialized data.

4.4 Addressing Modes

A microcontroller uses different addressing mechanisms similar to addressing mechanisms in referring to a person. A person may have a first and last name, nick name, Identification number for a country, organization etc. Similarly, memory location, register, port, peripheral etc may be addressed accordingly in different ways in computer systems.

An instruction is a command used to make the microcontroller perform an operation. It is composed of an Opcode and Operand. Opcode is the operation code that is performed. Ex: MOV, ADD etc. Based on the addressing mode and operation type addition letters are combined to form the final opcode like MOVLW, MOVF etc. The second part of an instruction is the operand which is optional and generally may contain an 8-bit data or register(s) or address(es) etc. EX: 0x34 or PORTB or 0xF81 etc. The operand used varies based on the type of addressing mode. The addressing modes employed in PIC18F458 are as follows.

Immediate Addressing

Register Addressing

Direct Addressing

Register Indirect

Indexed/Relative Addressing

Implicit Addressing

Immediate Addressing

Immediate addressing mode instructions specify the data value that forms the operand part of the instruction to perform an operation using the 8-bit data. In PIC18 such addressing mode instructions generally contain 'LW' as part of instruction's opcode. Given here are some examples for the same.

- MOVLW 0x23 → 0x23 value is moved to W-Reg
- MOVLW 8-Bit → An 8-Bit value is moved to W-Reg. 8-Bit indicates a value
- ADDLW 0x45 → 0x45H is added to W-Reg

Register Addressing

Register addressing mode instructions specify the name of the SFR registers that hold the data value. The name of the register forms the operand part of the instruction to perform an operation on the respective data in the register. Given here are some examples for the same.

- MOVF PORTB → The data value present in W-Reg is copied to PORTB,

where PORTB is the name of the register, which also has a file location F81H (Non-Bank Mode) or 81H (Bank Mode) and can be addressed accordingly in Direct Addressing Mode.

- BCF STATUS, 0 → The 0th bit in STATUS register is cleared.

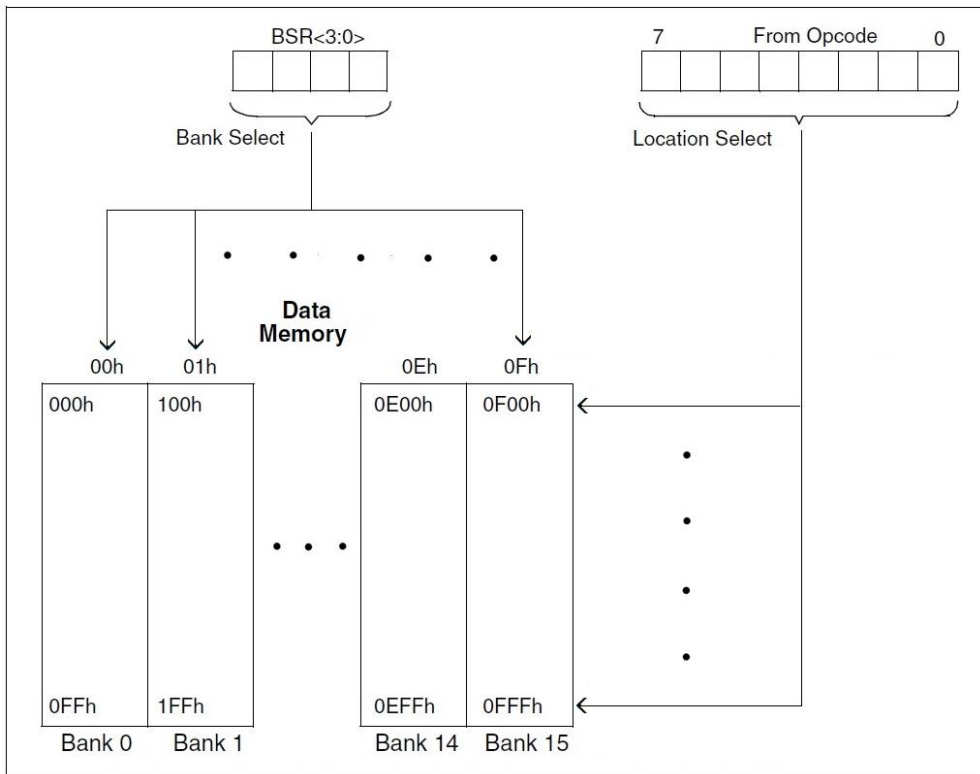


Fig 4.5 Direct Addressing mechanism

Direct Addressing

Direct addressing mode instructions specify the assigned address of memory, I/O, Peripheral or SFR's memory mapped address that hold the data value. The address forms the operand part of the instruction to perform an operation on the respective data in the given address. The address is an 8-Bit value in most instructions and 12-Bit value in instruction like MOVFF. Given here are some examples for the same.

- MOVF 0x55, 0 → The data value present in 0x55 is copied to W-Reg
- MOVWF 0x81, 0 → The data value present in W-Reg is copied to PORTB
- MOVF 0xFE8, 0xF81 → The data value present in W-Reg (0xFE8) is copied to PORTB

The Direct addressing relies on the use of access bit specified along with the instruction and the BSR-Bank Select Register bits to select access bank or one of the 16 banks of data memory. If the access bank bit 'a' is 0 access bank is selected, else the BSR bits 3-0 determine the bank number among the 16 banks.

Register Indirect Addressing

Register Indirect addressing mode instructions specify the name of the register that holds the address of memory, I/O, Peripheral or SFR's memory mapped address. The data value is present in address specified by the register. This implies that Instruction specifies address, and address contains the required data. i.e Register → Address → Data. The Register forms the operand part of the instruction to perform an operation on the respective data. These operations require FSR (FSRnH & FSRnL where n can take value 0, 1, 2 for PIC18) registers to hold the full 12-Bit RAM address & nonphysical registers like INDF to access data indirectly as part of instruction. Given here are some examples for the same.

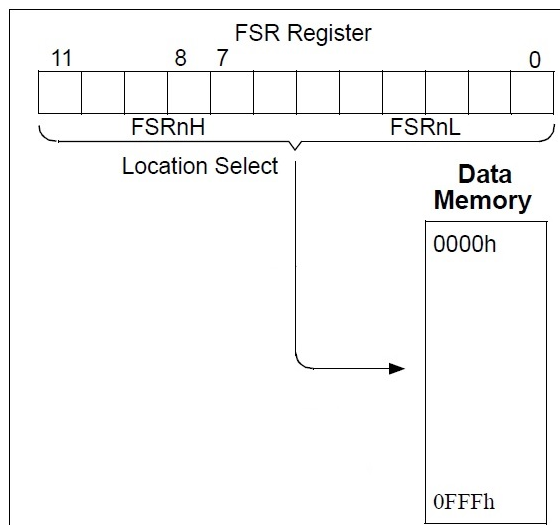


Fig 4.6 Indirect Addressing mechanism

- **MOVWF INDF0** → The INDF0 is not a physical register but is used as pointer. Actually address of RAM/Data Memory is held by FSR0. The instruction copies the data from W-Reg in to that address.
- **MOVF INDF, 0** → The data value present in address pointed as INDF0 is copied to W-Reg. FSR0 value remains unchanged.

Additionally, Table Read instructions TBLRD*, TBLWT* are used for reading & writing Tables in ROM operate using this addressing mode by making use of Table pointer and Table Latch.

Indexed Addressing

Indexed addressing mode is an extension of Register Indirect addressing mode. In addition to indirect addressing it allows for indexing features like Pre Increment, Post Increment, Post Decrement. These operations also require FSR registers to hold the address and the nonphysical registers like PREINC, POSTINC, POSTDEC need to be used to access data indirectly as part of instruction. Given here are some examples for the same.

- **MOVWF PREINC0** → Similar to INDF but the address is incremented first before performing the operation specified by instruction. i.e. address value in FSR0 is incremented, then data from W-Reg is copied in to the incremented address of FSR0. This is an indexed addressing mode
- **MOVWF POSTINC0** → Similar to INDF but the address is incremented after performing the operation specified by instruction. The data from W-Reg is copied in to the address pointed by FSR0 and then increments the address value in FSR0. This is an indexed addressing mode.
- **MOVWF POSTDEC0** → Similar to INDF but the address is decremented after performing the operation specified by instruction. The data from W-Reg is copied in to the address pointed by FSR0 and then decrements the address value in FSR0. This is an indexed addressing mode.

Additionally, Table Read instructions TBLRD+*, TBLRD*+, TBLRD*-, Table Write instructions TBLWT+*, TBLWT*+, TBLWT*- used for reading and writing Tables in ROM operate using indexed addressing mode by making use of Table pointer and Table Latch.

Relative Addressing

Relative addressing mode is an extension of Register Indirect addressing mode. In addition to indirect addressing it allows for relative addressing features of having a base value in a register and relative offset value in different register. These operations also require FSR registers to hold the address, W-Reg to hold the offset address value and the nonphysical register PLUSW need to be used to access data indirectly as part of instruction. Given here are some examples for the same.

- **CLRF PLUSW0** → Similar to INDF but the content of W-Reg are added to address value in FSR0. The value in address formed this addition is cleared as it is CLRF instruction. The WREG and FSR0 remain unchanged. This is a relative

addressing mode Where base value is taken from FSR0 and relative value from W-Reg.

- **MOVFF PLUSW0, PORTB** → Similar to INDF but the content of W-Reg are added to address value in FSR0. The value in address formed this addition is copied to PORTB as its MOVFF instruction. The WREG and FSR0 remain unchanged. This is a relative addressing mode Where base value is taken from FSR0 and relative value from W-Reg.

Implicit Addressing

Implicit addressing mode is a type of addressing mode where the instruction does not contain any operands. Given here are some examples for the same.

- **CLRWDT** → Clear Watchdog Timer
- **DAW** → Decimal Adjust W-Reg
- **NOP** → No Operation
- **SLEEP** → Make PIC18 to go into Sleep Mode

4.5 Instruction Set

Every computer is programmed to operate by commands. A certain group of commands are used for performing a required task or an application. These commands vary by functionality. In a microcontroller at the Assembly language level the commands are called as instructions. The set of instructions available in PIC18 microcontroller are grouped based on the type of functionality or the type of operations. The instruction set of PIC18F458 is classified as follows.

- Data Transfer Instructions
- Arithmetic Instructions
- Bit Wise Instructions
- Logical Instructions
- Branch & Control Instructions

A brief summary of instruction classification is presented here.

Data Transfer Instructions

The data transfer instructions are used for transferring of data. PIC18 contains RAM and ROM internally. A CPU performs an operation on the data that is available in data memory - RAM/SFRs. Similarly, some constant symbols are arranged in the form of a table and are generally stored in flash program memory. Hence data transfer instructions like mov, table read, table write, push, pop etc are required to transfer the data to and from data memory or the program memory.

Arithmetic Instructions

The arithmetic instructions are used to perform arithmetic operations like addition, subtraction, multiplication, increment & decrement etc. Microcontrollers based applications are required to process signal data to perform signal addition, subtraction etc. Hence arithmetic instructions are provided to process the signal data.

Logical Instructions

The logical instructions are used to perform logical operations like and, or, xor, rotate, compare, test etc. Microcontrollers based applications are required to perform conditional checking, masking to extract required data etc. Hence logical instructions are provided to logical processing of the available signal data.

Bitwise Instructions

The bit-wise instructions are used to perform bit-wise operations on specific bit(s) in a byte of data. Operations like setting, clearing toggling a bit or checking if a bit is set or reset etc. Microcontrollers based applications required to perform bit checking for reading/writing of I/O pin (activate or deactivate a control pin, led, actuator or read switch status) or data in register that may be sent or received from an I/O, etc. Hence bit-wise instructions are provided to perform bit wise operations than performing byte-wise masking and checking which is not efficient.

Branch & Control Instructions

The branch & control instructions are used to control the sequence of execution of instructions in CPU based on conditional requirements and data. Conditional, unconditional branch operations, calling and return from subroutine, control CPU state like sleep, reset etc. Microcontrollers based applications need to check for various conditions like I/O port status etc, and enable an actuator based on the port status which requires executing a part of the code based on conditions. Such features are provided through the branch instructions. Call and return are required for repeated execution of certain part of the code that is written as subroutines. Similarly, to save power and start from the beginning sleep and reset are used. Hence branch & control instructions are provided to control the branching, execution or control the CPU state.

Readers are encouraged to refer respective data sheets for the details of the instructions.

The instructions in the following tables make use of abbreviations representing a parameter. They are given below.

- k represents to constant literal 8-bit or 12-bit
- a represents to access mode,
 - 0 - default access bank
 - 1 - specific bank specified by BSR register

d	represents storage destination, 0 - W register 1 - file register
f	represents File register 8-bit
fs	represents to source file register 12 bit.
fd	represents to destination file register 12 bit.
b	represents bit in file register 0 to 7.
n	Relative address (2's Complement) for relative instructions Direct address for Call, Branch, Return instructions
s	Fast Call/Return mode select bit 0 – do not update into or from shadow register 1 – update into or from shadow register
C	Carry Flag
DC	Auxiliary/Digit Carry
Z	Zero Flag
OV	Overflow Flag
N	Sign/Negative Flag
<u>TO</u>	Time out bit
<u>PD</u>	Power Down bit

Data Memory Data Transfer Instructions					
Opcode	Operand	Description	Hex Code/ Instruction Word	No of Cycles	Status Affected
MOVLW	k	Move literal to WREG	'0000 1110 kkkk kkkk	1	None
MOVWF	f, a	Move WREG to f	'0110 111a ffff ffff	1	None
MOVF	f, d, a	Move f	'0101 00da ffff ffff	1	Z, N
MOVFF	fs, fd	Move fs (source) to 1st word fd (destination) 2nd word	'1100 ffff ffff ffff	2	None
			'1111 ffff ffff ffff		
MOVLB	k	Move literal to BSR<3:0>	'0000 0001 0000 kkkk	1	None
LFSR	f, k	Move literal (12-bit) 2nd word to FSRx 1st word	'1110 1110 00ff kkkk	2	None
			'1111 0000 kkkk kkkk		
Program Memory Data Transfer Instructions					
Opcode	Operand	Description	Hex Code/ Instruction Word	No of Cycles	Status Affected
TBLRD*		Table Read	'0000 0000 0000 1000	2	None
TBLRD*+		Table Read with post-increment	'0000 0000 0000 1001		None
TBLRD*-		Table Read with post-decrement	'0000 0000 0000 1010		None
TBLRD*+		Table Read with pre-increment	'0000 0000 0000 1011		None
TBLWT*		Table Write	'0000 0000 0000 1100	2 (5)	None
TBLWT*+		Table Write with post-increment	'0000 0000 0000 1101		None
TBLWT*-		Table Write with post-decrement	'0000 0000 0000 1110		None
TBLWT*+		Table Write with pre-increment	'0000 0000 0000 1111		None
Stack Operation					
Opcode	Operand	Description	Hex Code/ Instruction Word	No of Cycles	Status Affected
PUSH	—	Push top of return stack (TOS)	'0000 0000 0000 0101	1	None
POP	—	Pop top of return stack (TOS)	'0000 0000 0000 0110	1	None
Other					
Opcode	Operand	Description	Hex Code/ Instruction Word	No of Cycles	Status Affected
NOP	—	No Operation	'0000 0000 0000 0000	1	None
NOP	—	No Operation	'1111 xxxx xxxx xxxx	1	None

Logical Instructions					
Opcode	Operand	Description	Hex Code/ Instruction Word	No of Cycles	Status Affected
ANDLW	k	AND literal with WREG	'0000 1011 kkkk kkkk	1	Z, N
ANDWF	f, d, a	AND WREG with f	'0001 01da ffff ffff	1	Z, N
IORLW	k	Inclusive OR literal with WREG	'0000 1001 kkkk kkkk	1	Z, N
IORWF	f, d, a	Inclusive OR WREG with f	'0001 00da ffff ffff	1	Z, N
XORLW	k	Exclusive OR literal with WREG	'0000 1010 kkkk kkkk	1	Z, N
XORWF	f, d, a	Exclusive OR WREG with f	'0001 10da ffff ffff	1	Z, N
CLRF	f, a	Clear f	'0110 101a ffff ffff	1	Z
SETF	f, a	Set f	'0110 100a ffff ffff	1	None
COMF	f, d, a	Complement f	'0001 11da ffff ffff	1	Z, N
NEGF	f, a	Negate f	'0110 110a ffff ffff	1	C, DC, Z, OV, N
SWAPF	f, d, a	Swap nibbles in f	'0011 10da ffff ffff	1	None
RLCF	f, d, a	Rotate Left f through Carry	'0011 01da ffff ffff	1	C, Z, N
RLNCF	f, d, a	Rotate Left f (No Carry)	'0100 01da ffff ffff	1	Z, N
RRCF	f, d, a	Rotate Right f through Carry	'0011 00da ffff ffff	1	C, Z, N
RRNCF	f, d, a	Rotate Right f (No Carry)	'0100 00da ffff ffff	1	Z, N
Bit-wise Instructions					
Opcode	Operand	Description	Hex Code/ Instruction Word	No of Cycles	Status Affected
BCF	f, b, a	Bit Clear f	'1001 bbba ffff ffff	1	None
BSF	f, b, a	Bit Set f	'1000 bbba ffff ffff	1	None
BTG	f, d, a	Bit Toggle f	'0111 bbba ffff ffff	1	None
BTFSC	f, b, a	Bit Test f, Skip if Clear	'1011 bbba ffff ffff	1 (2 or 3)	None
BTFSS	f, b, a	Bit Test f, Skip if Set	'1010 bbba ffff ffff	1 (2 or 3)	None

Arithmetic Instructions					
Opcode	Operand	Description	Hex Code/ Instruction Word	No of Cycles	Status Affected
ADDLW	k	Add literal and WREG	'0000 1111 kkkk kkkk	1	C, DC, Z, OV, N
ADDWF	f, d, a	Add WREG and f	'0010 01da ffff ffff	1	C, DC, Z, OV, N
ADDWFC	f, d, a	Add WREG and Carry bit to f	'0010 00da ffff ffff	1	C, DC, Z, OV, N
DAW	—	Decimal Adjust WREG	'0000 0000 0000 0111	1	C
SUBLW	k	Subtract WREG from literal	'0000 1000 kkkk kkkk	1	C, DC, Z, OV, N
SUBWF	f, d, a	Subtract WREG from f	'0101 11da ffff ffff	1	C, DC, Z, OV, N
SUBFWB	f, d, a	Subtract f from WREG with borrow	'0101 01da ffff ffff	1	C, DC, Z, OV, N
SUBWFB	f, d, a	Subtract WREG from f with borrow	'0101 10da ffff ffff	1	C, DC, Z, OV, N
MULLW	k	Multiply literal with WREG	'0000 1101 kkkk kkkk	1	None
MULWF	f, a	Multiply WREG with f	'0000 001a ffff ffff	1	None
INCF	f, d, a	Increment f	'0010 10da ffff ffff	1	C, DC, Z, OV, N
DECF	f, d, a	Decrement f	'0000 01da ffff ffff	1	C, DC, Z, OV, N
Mixed Instructions					
Opcode	Operand	Description	Hex Code/ Instruction Word	No of Cycles	Status Affected
INCFSZ	f, d, a	Increment f, Skip if 0	'0011 11da ffff ffff	1 (2 or 3)	None
INFSNZ	f, d, a	Increment f, Skip if Not 0	'0100 10da ffff ffff	1 (2 or 3)	None
DECFSZ	f, d, a	Decrement f, Skip if 0	'0010 11da ffff ffff	1 (2 or 3)	None
DCFSNZ	f, d, a	Decrement f, Skip if Not 0	'0100 11da ffff ffff	1 (2 or 3)	None
TSTFSZ	f, a	Test f, skip if 0	'0110 011a ffff ffff	1 (2 or 3)	None
CPFSEQ	f, a	Compare f with WREG, skip =	'0110 001a ffff ffff	1 (2 or 3)	None
CPFSGT	f, a	Compare f with WREG, skip >	'0110 010a ffff ffff	1 (2 or 3)	None
CPFSLT	f, a	Compare f with WREG, skip <	'0110 000a ffff ffff	1 (2 or 3)	None

Branch & Control Instructions					
Opcode	Operand	Description	Hex Code/ Instruction Word	No of Cycles	Status Affected
BC	n	Branch if Carry	'1110 0010 nnnn nnnn	1 (2)	None
BNC	n	Branch if Not Carry	'1110 0011 nnnn nnnn	1 (2)	None
BN	n	Branch if Negative	'1110 0110 nnnn nnnn	1 (2)	None
BNN	n	Branch if Not Negative	'1110 0111 nnnn nnnn	1 (2)	None
BOV	n	Branch if Overflow	'1110 0100 nnnn nnnn	1 (2)	None
BN OV	n	Branch if Not Overflow	'1110 0101 nnnn nnnn	1 (2)	None
BZ	n	Branch if Zero	'1110 0000 nnnn nnnn	1 (2)	None
BNZ	n	Branch if Not Zero	'1110 0001 nnnn nnnn	1 (2)	None
BRA	n	Branch Unconditionally	'1101 0nnn nnnn nnnn	2	None
GOTO	n	Go to address 1st word 2nd word	'1110 1111 kkkk kkkk '1111 kkkk kkkk kkkk	2	None
CALL	n, s	Call subroutine 1st word 2nd word	'1110 110s kkkk kkkk '1111 kkkk kkkk kkkk	2	None
RCALL	n	Relative Call	'1101 1nnn nnnn nnnn	2	None
RETURN	s	Return from Subroutine	'0000 0000 0001 001s	2	None
RETLW	k	Return with literal in WREG	'0000 1100 kkkk kkkk	2	None
RETFIE	s	Return from interrupt enable	'0000 0000 0001 000s	2	GIE/GIEH, PEIE/GIEL
RESET	—	Software device Reset	'0000 0000 1111 1111	1	All
CLRWD T	—	Clear Watchdog Timer	'0000 0000 0000 0100	1	TO, PD
SLEEP	—	Go into Standby mode	'0000 0000 0000 0011	1	TO, PD

Instruction Description

Data memory - Data Transfer Instructions

MOVLW: Move 8-bit Literal k to W-Register

Format & Ex: MOVLW k MOVLW 0x80 --> W = 0x80

Flags affected: None

MOVWF: Move W-Register content to File Register

Format & Ex: MOVWF f, a

Assuming

WREG content = 0x23 and BSR = 2

MOVWF 0x10, 0 --> [0x010] = 0x23

MOVWF 0x10, 1 --> [0x210] = 0x23

Flags affected: None

MOVF: Move File Register content to W-Register or back to same File Register.

Format & Ex: MOVF f, d, a

Assuming

File Register content = 0x23, W-Register content = 0x12 & BSR = 2

MOVF 0x10, 0, 0 --> [0x010] = 0x23 W-Reg = 0x23

MOVF 0x10, 1, 0 --> [0x010] = 0x23 W-Reg = 0x12

MOVF 0x10, 0, 1 --> [0x210] = 0x23 W-Reg = 0x23

MOVF 0x10, 1, 1 --> [0x210] = 0x23 W-Reg = 0x12

Flags affected: Z, N Flags

MOVFF: Move 12-bit File Register fs content to 12-bit File Register fd.

Format & Ex: MOVFF fs, fd

Assuming

Source File Register fs 0x100 content = 0x12,

Destination File Register fd 0x200 content = 0x45

MOVFF 0x100, 0x200 --> [0x100] = 0x12 [0x200] = 0x12

MOVFF 0x200, 0x100 --> [0x100] = 0x45 [0x200] = 0x45

Flags affected: None

MOVLB: Move 4-bit literal to Low Nibble of BSR register.

Format & Ex: MOVLB k

Assuming

BSR = 1

MOVLB 2

--> BSR<3:0> = 2

Flags affected: None

LFSR: Load File Select Register - FSR 0, 1 or 2 with 12-bit Literal address.

Format & Ex: LFSR f, k

Assuming

LFSR 0, 0x100

--> FSR0 = 0x100

LFSR 1, 0x200

--> FSR0 = 0x200

LFSR 2, 0x300

--> FSR0 = 0x300

Flags affected: None

Program memory Data Transfer Instructions

Consider Flash Memory Table data containing Gray Code starting at location 080000H. Consider each type of instruction refers to this table without modification. TBLPTR – Table Pointer Register is 21-bit (TBLPTRU-TBLPTRH-TBLPTRL) and TABLAT – Table Latch Register is 8-bit.

<u>Decimal</u>	<u>Binary</u>	<u>Gray</u>	<u>Location</u>
0	0000	0000	080000H
1	0001	0001	080001H
2	0010	0011	080002H
3	0011	0010	080003H
4	0100	0110	080004H
5	0101	0111	080005H
6	0110	0101	080006H
7	0111	0100	080007H

TBLRD*: Read a byte from Table in Flash memory pointed by TBLPTR - Table Pointer Register into TABLAT - Table Latch Register.

Format & Ex: TBLRD*

Assuming data shown above in Flash Memory Table Data

TBLPTR = 080002H

TBLRD* --> TABLAT = 0x03

--> TBLPTR = 080002H

TBLPTR = 080003H

TBLRD* --> TABLAT = 0x02

--> TBLPTR = 080003H

Flags affected: None

TBLRD*+: Read a byte from Table in Flash memory pointed by TBLPTR - Table Pointer Register into TABLAT - Table Latch Register, after table read, Increment TBLPTR by 1, i.e. Post Increment.

Format & Ex: TBLRD*+

Assuming data shown above in Flash Memory Table Data

TBLPTR = 080002H

TBLRD*+ --> TABLAT = 0x03 --> TBLPTR = 080003H

TBLPTR = 080005H

TBLRD*+ --> TABLAT = 0x07 --> TBLPTR = 080006H

Flags affected: None

TBLRD*-: Read a byte from Table in Flash memory pointed by TBLPTR - Table Pointer Register into TABLAT - Table Latch Register, after table read, Decrement TBLPTR by 1, i.e. Post Decrement.

Format & Ex: TBLRD*-

Assuming data shown above in Flash Memory Table Data

TBLPTR = 080002H

TBLRD*- --> TABLAT = 0x03 --> TBLPTR = 080001H

TBLPTR = 080005H

TBLRD*- --> TABLAT = 0x07 --> TBLPTR = 080004H

Flags affected: None

TBLRD+*: Increment TBLPTR by 1, i.e. Pre Increment, then read a byte from Table in Flash memory pointed by TBLPTR - Table Pointer Register into TABLAT - Table Latch Register

Format & Ex: TBLRD+*

Assuming data shown above in Flash Memory Table Data

TBLPTR = 080002H

TBLRD+* --> TBLPTR = 080003H --> TABLAT = 0x02

TBLPTR = 080005H

TBLRD+* --> TBLPTR = 080006H --> TABLAT = 0x05

Flags affected: None

TBLWT*: Write a data byte present in TABLAT - Table Latch Register into the Table in Flash memory pointed by TBLPTR - Table Pointer Register.

Format & Ex: TBLWT*

Assuming data shown above in Flash Memory Table Data

TBLPTR = 080002H TABLAT = 0x02

TBLWT* --> [080002H] = 0x02 --> TBLPTR = 080002H

TBLPTR = 080005H TABLAT = 0x05

TBLWT* --> [080005H] = 0x05 --> TBLPTR = 080005H

Flags affected: None

TBLWT*+: Write a data byte present in TABLAT - Table Latch Register into the Table in Flash memory pointed by TBLPTR - Table Pointer Register, then Increment TBLPTR by 1, i.e. Post increment.

Format & Ex: TBLWT*+

Assuming data shown above in Flash Memory Table Data

TBLPTR = 080002H TABLAT = 0x02

TBLWT* + --> [080002H] = 0x02 --> TBLPTR = 080003H

TBLPTR = 080005H TABLAT = 0x05

TBLWT* + --> [080005H] = 0x05 --> TBLPTR = 080006H

Flags affected: None

TBLWT*-: Write a data byte present in TABLAT - Table Latch Register into the Table in Flash memory pointed by TBLPTR - Table Pointer Register, then Decrement TBLPTR by 1, i.e. Post Decrement.

Format & Ex: TBLWT*-

Assuming data shown above in Flash Memory Table Data

TBLPTR = 080002H TABLAT = 0x02

TBLWT* - --> [080002H] = 0x02 --> TBLPTR = 080001H

TBLPTR = 080005H TABLAT = 0x05

TBLWT* - --> [080005H] = 0x05 --> TBLPTR = 080004H

Flags affected: None

TBLWT+*: Increment TBLPTR by 1, i.e. Pre Increment, then write a data byte present in TABLAT - Table Latch Register into the Table in Flash memory pointed by TBLPTR - Table Pointer Register.

Format & Ex: TBLWT+*

Assuming data shown above in Flash Memory Table Data

TBLPTR = 080002H TABLAT = 0x03

TBLWT+* --> TBLPTR = 080003H --> [080003H] = 0x03

TBLPTR = 080005H TABLAT = 0x07

TBLWT+*

--> TBLPTR = 080006H

--> [080006H] = 0x07

Flags affected: None

Stack Operation Instructions

While an instruction is being executed, the PC – Program Counter contains the address PC + 2 (i.e. generally points to next instruction). The Stack in PIC18 is a maximum of 31 level and changes by a level for a PUSH or POP instruction. The 21-bit TOS - TOP of Stack Register (TOSU-TOSH-TOSL) contains the value at level pointed by Stack Pointer – STKPTR<4:0>. Stack levels are from 1 to 31.

PUSH: Push the PC – Program Counter value on to the Stack. The TOS register will contain the value of this address. I.e. While PUSH instruction is executed, the PC contains PC +2. This is stored into Stack. For every PUSH, the Level of the Stack is incremented by 1.

Assuming PUSH instruction is at 040000H while PUSH instruction is executing, PC contains 040002H. If STKPTR<4:0> is at 3 the value of 040002H is stored at level 4 and the TOS register contains the value of 040002H and STKPTR<4:0> becomes 4.

i.e. Before PUSH instruction

TOS = 111111H PC = 040000H STKPTR<4:0> = 3

i.e. During PUSH instruction execution

TOS = 111111H PC = 040002H

i.e. After PUSH instruction execution

TOS = 040002H PC = 040002H STKPTR<4 : 0> = STKPTR<4 : 0> + 1 = 4

Flags affected: None

POP: POP the PC – Program Counter value from the Stack. The TOS register will contain the value of this address. I.e. While POP instruction is executed, the PC contains PC +2. This is discarded and the TOS value of the Stack is copied to PC. For every POP, the Level of the Stack is decremented by 1.

Assuming POP instruction is at 040008H while POP instruction is executing, PC contains 04000AH. If STKPTR<4:0> is at 4 the value of 04000AH is discarded and PC is loaded with value stored at level 4 i.e the TOS register contains the value of 040002H and STKPTR<4:0> becomes 3.

i.e. Before PUSH instruction

TOS = 0400002H PC = 040008H STKPTR<4:0> = 4

i.e. During PUSH instruction execution

TOS = 040002H PC = 04000AH

i.e. After PUSH instruction execution

TOS = 111111H PC = 040002H STKPTR<4 : 0> = STRKPTR<4 : 0> - 1 = 3
 Flags affected: None

Other Instructions

NOP: No Operations. Performs no operation. Just acts as a delay.
 Flags affected: None

Logical Instructions

ANDLW: Perform AND operation between the 8-bit Literal and 8-bit value in W-Register and store in W-Register.

Format & Ex: ANDLW k

Assuming W-Register = 0x45

ANDLW 0xF4

--> W-Register = 0xF4. 0x45 = 0x44

1111	0100	0xF4
<u>0100</u>	<u>0101</u>	<u>0x45</u>
0100	0100	0x44

Flags affected: Z, N

ANDWF: Perform AND operation between the 8-bit value in File Register and 8-bit value in W-Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: ANDWF f, d, a

Assuming W-Register = 0x45,

[0x10] = 0xF4, BSR = 2

ANDWF 0x10, 0, 0

--> W-Register = [0x10]. 0x45

= 0xF4. 0x45 = 0x44

Assuming W-Register = 0x45,

[0x10] = 0xF4, BSR = 2

ANDWF 0x10, 1, 0

--> [0x10] = [0x10]. 0x45

= 0xF4. 0x45 = 0x44

Assuming W-Register = 0x65,

[0x210] = 0x23, BSR = 2

ANDWF 0x10, 0, 2

--> W-Register = [0x210]. 0x65

= 0x23. 0x65 = 0x21

Assuming W-Register = 0x65,

[0x210] = 0x23, BSR = 2

ANDWF 0x10, 1, 2

--> [0x210] = [0x210]. 0x65

= 0x23. 0x65 = 0x21

Flags affected: Z, N

IORLW: Perform OR operation between the 8-bit Literal and 8-bit value in W-Register and store in W-Register.

Format & Ex: IORLW k

Assuming W-Register = 0x45

IORLW 0xF4

--> W-Register = 0xF4 | 0x45 = 0xF5

1111	0100	0xF4
<u>0100</u>	<u>0101</u>	<u>0x45</u>
1111	0101	0xF5

Flags affected: Z, N

IORWF: Perform OR operation between the 8-bit value in File Register and 8-bit value in W-Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: IORWF f, d, a

Assuming W-Register = 0x45, [0x10] = 0xF4, BSR = 2

IORWF 0x10, 0, 0 --> W-Register = [0x10] | 0x45

= 0xF4 | 0x45 = 0xF5

Assuming W-Register = 0x45, [0x10] = 0xF4, BSR = 2

IORWF 0x10, 1, 0 --> [0x10] = [0x10] | 0x45

= 0xF4 | 0x45 = 0xF5

Assuming W-Register = 0x65, [0x210] = 0x23, BSR = 2

IORWF 0x10, 0, 2 --> W-Register = [0x210] | 0x65

= 0x23 | 0x65 = 0x67

Assuming W-Register = 0x65, [0x210] = 0x23, BSR = 2

IORWF 0x10, 1, 2 --> [0x210] = [0x210] | 0x65

= 0x23 | 0x65 = 0x67

Flags affected: Z, N

XORLW: Perform XOR operation between the 8-bit Literal and 8-bit value in W-Register and store in W-Register.

Format & Ex: XORLW k

Assuming W-Register = 0x45

XORLW 0xF4

--> W-Register = 0xF4 $\oplus$ 0x45 = 0xF5

1111	0100	0xF4
<u>0100</u>	<u>0101</u>	<u>0x45</u>
1011	0001	0xB1

Flags affected: Z, N

XORWF: Perform XOR operation between the 8-bit value in File Register and 8-bit value in W-Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: XORWF f, d, a

Assuming W-Register = 0x45,	[0x10] = 0xF4, BSR = 2
XORWF 0x10, 0, 0	--> W-Register = [0x10] $\oplus$ 0x45
	= 0xF4 $\oplus$ 0x45 = 0xB1
Assuming W-Register = 0x45,	[0x10] = 0xF4, BSR = 2
XORWF 0x10, 1, 0	--> [0x10] = [0x10] $\oplus$ 0x45
	= 0xF4 $\oplus$ 0x45 = 0xB1
Assuming W-Register = 0x65,	[0x210] = 0x23, BSR = 2
XORWF 0x10, 0, 2	--> W-Register = [0x210] $\oplus$ 0x65
	= 0x23 $\oplus$ 0x65 = 0x46
Assuming W-Register = 0x65,	[0x210] = 0x23, BSR = 2
XORWF 0x10, 1, 2	--> [0x210] = [0x210] $\oplus$ 0x65
	= 0x23 $\oplus$ 0x65 = 0x46

Flags affected: Z, N

CLRF: Clear the 8-bit data value of File Register to 0's, i.e. content of the File Register of default access bank or one pointed by BSR Register based on the choice is cleared to 0.

Format & Ex: CLRF f, a

Assuming [0x20] = 0x45, BSR = 1	
CLRF 0x20, 0	--> [0x20] = 0x00
Assuming [0x20] = 0x45, BSR = 1	
CLRF 0x20, 1	--> [0x120] = 0x00

Flags affected: Z

SETF: Set the 8-bit data value of File Register to 1's, i.e. content of the File Register of default access bank or one pointed by BSR Register based on the choice is set to 1's.

Format & Ex: SETF f, a

Assuming [0x20] = 0x45, BSR = 1	
SETF 0x20, 0	--> [0x20] = 0xFF
Assuming [0x20] = 0x45, BSR = 1	
SETF 0x20, 1	--> [0x120] = 0xFF

Flags affected: None

COMF: Complement the 8-bit data value of File Register, 1's to 0's and 0's to 1's. i.e. content of the File Register of default access bank or one pointed by BSR Register based on the choice is toggled.

Format & Ex: COMF f, d, a

Assuming [0x20] = 0x45, BSR = 1

COMF 0x20, 0 --> [0x20] = 0xBA

Assuming [0x20] = 0x45, BSR = 1

COMF 0x20, 1 --> [0x120] = 0xBA

Flags affected: Z, N

NEGF: Get the 8-bit data value in File Register, store the 2's complement value of it in the same File Register. i.e. content of the File Register of default access bank or one pointed by BSR Register based on the choice is 2's complemented.

Format & Ex: NEGF f, a

Assuming [0x20] = 0x45, BSR = 1

NEGF 0x20, 0 --> [0x20] = 0xBB

Assuming [0x20] = 0x45, BSR = 1

NEGF 0x20, 1 --> [0x120] = 0xBB

Flags affected: All

SWAPF: Swap the nibbles of the 8-bit data value in File Register, store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: SWAPF f, d, a

Assuming [0x20] = 0x45, BSR = 1

SWAPF 0x20, 0, 0 --> W-Register = 0x54

Assuming [0x20] = 0x45, BSR = 1

SWAPF 0x20, 1, 0 --> [0x20] = 0x54

Assuming [0x20] = 0x45, BSR = 1

SWAPF 0x20, 0, 1 --> W-Register = 0x54

Assuming [0x20] = 0x45, BSR = 1

SWAPF 0x20, 1, 1 --> [0x120] = 0x54

Flags affected: None

RLCF: Rotate the 8-bit value present in the corresponding File Register to the left through Carry and store the result in the W-Register or the File Register of default access bank or one pointed by BSR Register based on the choice. $f<n> \rightarrow rd<n+1>; f<7> \rightarrow C; C \rightarrow rd<0>$. I.e All the bits in f register are moved left, MSB bit $f<7>$ goes to C, Value of C is copied to LSB of destination register $rd<0>$. The RLCF operation is shown in Fig 4.7a

Format & Ex: RLCF f, d, a

Assuming W-Register = 0x30, BSR = 1, [0x20] = 0x11, [0x120] = 0x88

C = 0

RLCF 0x20, 0, 0	--> W-Register = 0x22, C = 0
RLCF 0x20, 1, 0	--> [0x20] = 0x22, C = 0
RLCF 0x20, 0, 1	--> W-Register = 0x10, C = 1
RLCF 0x20, 1, 1	--> [0x120] = 0x10, C = 1

C = 1

RLCF 0x20, 0, 0	--> W-Register = 0x23, C = 0
RLCF 0x20, 1, 0	--> [0x20] = 0x23, C = 0
RLCF 0x20, 0, 1	--> W-Register = 0x11, C = 1
RLCF 0x20, 1, 1	--> [0x120] = 0x11, C = 1

Flags affected: C, Z, N

RLNCF: Rotate the 8-bit value present in the corresponding File Register to the left without Carry and store the result in the W-Register or the File Register of default access bank or one pointed by BSR Register based on the choice. $f<n> \rightarrow rd<n+1>; f<7> \rightarrow rd<0>$. I.e All the bits in f register are moved left, MSB bit $f<7>$ is copied to LSB of destination register $rd<0>$. The RLNCF operation is shown in Fig 4.7b

Format & Ex: RLNCF f, d, a

Assuming W-Register = 0x30, BSR = 1, [0x20] = 0x11, [0x120] = 0x88

RLNCF 0x20, 0, 0	--> W-Register = 0x22
RLNCF 0x20, 1, 0	--> [0x20] = 0x22
RLNCF 0x20, 0, 1	--> W-Register = 0x11
RLNCF 0x20, 1, 1	--> [0x120] = 0x11

Flags affected: Z, N

RRCF: Rotate the 8-bit value present in the corresponding File Register to the right through Carry and store the result in the W-Register or the File Register of default access bank or one pointed by BSR Register based on the choice. $f<n> \rightarrow rd<n-1>; f<0> \rightarrow C; C \rightarrow rd<7>$. I.e All the bits are moved right, LSB bit $f<0>$ goes to C, Value of C is copied to MSB of destination register $rd<7>$. The RRCF operation is shown in Fig 4.7c

Format & Ex: RRCF f, d, a

Assuming W-Register = 0x30, BSR = 1, [0x20] = 0x11, [0x120] = 0x88

C = 0

RRCF 0x20, 0, 0	--> W-Register = 0x08, C = 1
RRCF 0x20, 1, 0	--> [0x20] = 0x08, C = 1
RRCF 0x20, 0, 1	--> W-Register = 0x44, C = 0
RRCF 0x20, 1, 1	--> [0x120] = 0x44, C = 0

C = 1

RRCF 0x20, 0, 0	--> W-Register = 0x88, C = 1
RRCF 0x20, 1, 0	--> [0x20] = 0x88, C = 1
RRCF 0x20, 0, 1	--> W-Register = 0xC4, C = 0
RRCF 0x20, 1, 1	--> [0x120] = 0xC4, C = 0

Flags affected: C, Z, N

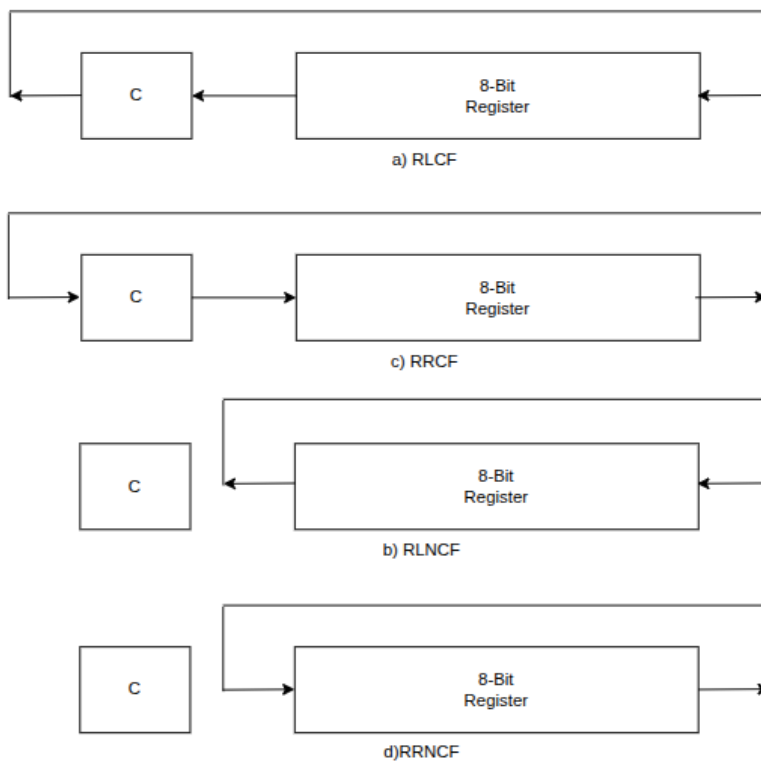


Fig 4.7 Rotate Operations

RRNCF: Rotate the 8-bit value present in the corresponding File Register to the right without Carry and store the result in the W-Register or the File Register of default access bank or one

pointed by BSR Register based on the choice. $f<n> \rightarrow rd<n-1>$; $f<0> \rightarrow rd<7>$. I.e All the bits are moved right, LSB bit $f<0>$ is copied to MSB of destination register $rd<7>$. The RRNCF operation is shown in Fig 4.7d

Format & Ex: RRNCF f, d, a

Assuming W-Register = 0x30, BSR = 1, [0x20] = 0x11, [0x120] = 0x88

RRNCF 0x20, 0, 0 --> W-Register = 0x88

RRNCF 0x20, 1, 0 --> [0x20] = 0x88

RRNCF 0x20, 0, 1 --> W-Register = 0x44

RRNCF 0x20, 1, 1 --> [0x120] = 0x44

Flags affected: Z, N

Arithmetic Instructions

ADDLW: Add the 8-bit Literal to 8-bit value in W-Register and store in W-Register.

Format & Ex: ADDLW k

Assuming W-Register = 0x45

ADDLW 0x20 --> W-Register = 0x20 + 0x45 = 0x65

Flags affected: All Examples in 5th Chapter, demonstrated with programs

ADDWF: Add the 8-bit value in File Register to 8-bit value in W-Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: ADDWF f, d, a

Assuming W-Register = 0x45, [0x10] = 0x20, BSR = 1

`ADDWF 0x10, 0, 0` --> W-Register = $[0x10] + 0x45$

$= 0x20 + 0x45 = 0x65$

Assuming W-Register = 0x45, [0x10] = 0x20, BSR = 1

`ADDWF 0x10, 1, 0` --> $[0x10] = [0x10] + 0x45$

$= 0x20 + 0x45 = 0x65$

Assuming W-Register = 0x45, [0x110] = 0x30, BSR = 1

`ADDWF 0x10, 0, 1` --> W-Register = [0x110] + 0x45
 = 0x30 + 0x45 = 0x75

Assuming W-Register = 0x45, [0x110] = 0x30, BSR = 1

`ADDWF 0x10, 1, 1` --> $[0x110] = [0x110] + 0x45$

$= 0x30 + 0x45 = 0x75$

Flags affected: All Examples in 5th Chapter, demonstrated with programs

ADDWFC: Add the 8-bit value in File Register to 8-bit value in W-Register and Carry, store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: ADDWFC f, d, a

Assuming W-Register = 0x45,	[0x10] = 0x20,	BSR = 1,	C = 1
ADDWFC 0x10, 0, 0	--> W-Register = [0x10] + 0x45 + 1		
	= 0x20 + 0x45 + 1 = 0x66		
Assuming W-Register = 0x45,	[0x10] = 0x20,	BSR = 1,	C = 1
ADDWFC 0x10, 1, 0	--> [0x10] = [0x10] + 0x45 + 1		
	= 0x20 + 0x45 + 1 = 0x66		
Assuming W-Register = 0x45,	[0x110] = 0x30,	BSR = 1,	C = 1
ADDWFC 0x10, 0, 1	--> W-Register = [0x110] + 0x45 + 1		
	= 0x30 + 0x45 + 1 = 0x76		
Assuming W-Register = 0x45,	[0x110] = 0x30,	BSR = 1,	C = 1
ADDWFC 0x10, 1, 1	--> [0x110] = [0x110] + 0x45 + 1		
	= 0x30 + 0x45 + 1 = 0x76		

Flags affected: All Examples in 5th Chapter, demonstrated with programs

DAW: Decimal Adjust W-Register after an Addition operation. I.e Convert to Decimal Value - Packed BCD format after an addition operation. Any digit > 9 (i.e A-F) is converted to range of 0 – 9 by adding a 6 to it.

For lower digit DC = 1 or W<3:0> > 9 --> W<3:0> + 6; else W<3:0>

For higher digit C = 1 or W<7:4> > 9 --> W<7:4> + 6 else W<7:4>

Format & Ex: DAW

Assuming W-Register = 0x30,	[0x10] = 0x20,	BSR = 1,	C = 1
ADDWFC 0x10, 0, 0	--> W-Register = [0x10] + 0x30 + 1		
	= 0x20 + 0x30 + 1 = 0x50		
	C = 0		
DAW	--> W-Register = 80d		

Flags affected: C; Set when result > packed BCD i.e > 99d i.e >= 0x64

SUBLW: Subtract the 8-bit value in W-Register from 8-bit Literal and store in W-Register.

Format & Ex: SUBLW k

Assuming W-Register = 0x20	
SUBLW 0x45	--> W-Register = 0x45 - 0x20 = 0x25
Assuming W-Register = 0x45	
SUBLW 0x20	--> W-Register = 0x20 - 0x45 = - 0x25

Flags affected: All Examples in 5th Chapter, demonstrated with programs

SUBWF: Subtract the 8-bit value in W-Register from 8-bit value in File Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: SUBWF f, d, a

Assuming W-Register = 0x20,	[0x10] = 0x45, BSR = 1
SUBWF 0x10, 0, 0	--> W-Register = [0x10] - 0x20
	= 0x45 - 0x20 = 0x25
Assuming W-Register = 0x45,	[0x10] = 0x20, BSR = 1
SUBWF 0x10, 1, 0	--> [0x10] = [0x10] - 0x45
	= 0x20 - 0x45 = - 0x25
Assuming W-Register = 0x30,	[0x110] = 0x45, BSR = 1
SUBWF 0x10, 0, 1	--> W-Register = [0x110] - 0x30
	= 0x45 - 0x30 = 0x15
Assuming W-Register = 0x45,	[0x110] = 0x30, BSR = 1
SUBWF 0x10, 1, 1	--> [0x110] = [0x110] - 0x45
	= 0x30 - 0x45 = - 0x15

Flags affected: All Examples in 5th Chapter, demonstrated with programs

SUBWFB: Subtract the 8-bit value in W-Register from 8-bit value in File Register, also subtract inverse of Carry and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: SUBWFB f, d, a

Assuming W-Register = 0x20,	[0x10] = 0x45, BSR = 1, C = 1
SUBWFB 0x10, 0, 0	--> W-Register = [0x10] - 0x20 - 0
	= 0x45 - 0x20 - 0 = 0x25
Assuming W-Register = 0x45,	[0x10] = 0x20, BSR = 1, C = 0
SUBWFB 0x10, 1, 0	--> [0x10] = [0x10] - 0x45 - 1
	= 0x20 - 0x45 - 1 = - 0x26
Assuming W-Register = 0x30,	[0x110] = 0x45, BSR = 1, C = 1
SUBWFB 0x10, 0, 1	--> W-Register = [0x110] - 0x30 - 0
	= 0x45 - 0x30 - 0 = 0x15
Assuming W-Register = 0x45,	[0x110] = 0x30, BSR = 1, C = 0
SUBWFB 0x10, 1, 1	--> [0x110] = [0x110] - 0x45 - 1
	= 0x30 - 0x45 - 1 = - 0x16

Flags affected: All Examples in 5th Chapter, demonstrated with programs

SUBFWB: Subtract the 8-bit value in File Register from 8-bit value in W-Register, also subtract inverse of Carry and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: SUBFWB f, d, a

Assuming W-Register = 0x20,	[0x10] = 0x45,	BSR = 1,	C = 1
SUBFWB 0x10, 0, 0	--> W-Register = 0x20 - [0x10] - 0		
	= 0x20 - 0x45 - 0 = - 0x25		
Assuming W-Register = 0x45,	[0x10] = 0x20,	BSR = 1,	C = 0
SUBFWB 0x10, 1, 0	--> [0x10] = 0x45 - [0x10] - 1		
	= 0x45 - 0x20 - 1 = 0x24		
Assuming W-Register = 0x30,	[0x110] = 0x45,	BSR = 1,	C = 1
SUBFWB 0x10, 0, 1	--> W-Register = 0x30 - [0x110] - 0		
	= 0x30 - 0x45 - 0 = - 0x15		
Assuming W-Register = 0x45,	[0x110] = 0x30,	BSR = 1,	C = 0
SUBFWB 0x10, 1, 1	--> [0x110] = 0x45 - [0x110] - 1		
	= 0x45 - 0x30 - 1 = 0x14		

Flags affected: All Examples in 5th Chapter, demonstrated with programs

MULLW: Multiply the 8-bit value in W-Register with 8-bit literal value and store the result in PRODH and PRODL registers.

Format & Ex: MULLW k

Assuming W-Register = 0x20	
MULLW 0x10	--> PRODH: PRODL = W-Register * 0x10
	= 0x20 * 0x10 = 0x0200
	PRODH = 0x20, PRODL = 0x00

Flags affected: None

MULWF: Multiply the 8-bit value in W-Register with 8-bit value in File Register and store the result in PRODH and PRODL registers.

Format & Ex: MULWF f, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x10	
MULWF 0x30, 0	--> PRODH: PRODL = W-Register * [0x10]
	= 0x20 * 0x10 = 0x0200
	PRODH = 0x20, PRODL = 0x00
Assuming W-Register = 0x20, BSR = 1, [0x130] = 0x10	
MULWF 0x30, 1	--> PRODH: PRODL = W-Register * [0x130]
	= 0x20 * 0x10 = 0x0200

PRODH = 0x20, PRODL = 0x00

Flags affected: None

INCF: Increment the 8-bit value present in File Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: INCF f, d, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x10, [0x130] = 0x40

INCF 0x30, 0, 0	--> W-Register = [0x30] + 1 = 0x10 + 1 = 0x11
INCF 0x30, 1, 0	--> [0x30] = [0x30] + 1 = 0x10 + 1 = 0x11
INCF 0x30, 0, 1	--> W-Register = [0x130] + 1 = 0x40 + 1 = 0x41
INCF 0x30, 1, 1	--> [0x130] = [0x130] + 1 = 0x40 + 1 = 0x41

Flags affected: All

DECF: Decrement the 8-bit value present in File Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice.

Format & Ex: DECF f, d, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x10, [0x130] = 0x40

DECF 0x30, 0, 0	--> W-Register = [0x30] - 1 = 0x10 - 1 = 0x0F
DECF 0x30, 1, 0	--> [0x30] = [0x30] - 1 = 0x10 - 1 = 0x0F
DECF 0x30, 0, 1	--> W-Register = [0x130] - 1 = 0x40 - 1 = 0x3F
DECF 0x30, 1, 1	--> [0x130] = [0x130] - 1 = 0x40 - 1 = 0x3F

Flags affected: All

Bit-wise Instructions

BCF: Clear the corresponding bit in the respective File Register.

Format & Ex: BCF f, b, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x8F, [0x130] = 0xF8

BCF 0x30, 3, 0	--> [0x30] = 0x87 Bit 3 --> 0
BCF 0x30, 4, 1	--> [0x130] = 0xE8 Bit 4 --> 0

Flags affected: None

BSF: Set the corresponding bit in the respective File Register.

Format & Ex: BSF f, b, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x70, [0x130] = 0x0F

BSF 0x30, 3, 0 --> [0x30] = 0x71 Bit 3 --> 1

BSF 0x30, 4, 1 --> [0x130] = 0x1F Bit 4 --> 1

Flags affected: None

BTG: Toggle the corresponding bit in the respective File Register.

Format & Ex: BTG f, b, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x8F, [0x130] = 0x0F

BTG 0x30, 3, 0 --> [0x30] = 0x87 Bit 3 --> 0

BTG 0x30, 4, 1 --> [0x130] = 0x1F Bit 4 --> 1

Flags affected: None

BTFSC: Test the corresponding bit in the respective File Register and skip the next instruction if the bit is clear. Discard the next instruction that is fetched and execute NOP if the bit is 0, else execute next instruction.

Format & Ex: BTFSC f, b, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x8F, [0x130] = 0x0F

Instruction Sequence is

BTFSC

BSF

MOLW

BTFSC 0x30, 3, 0 --> Bit 3 is 1 in 0x8F,

So, condition not satisfied,

BSF and MOLW are executed.

BTFSC 0x30, 4, 1 --> Bit 4 is 0 in 0x0F,

So, condition satisfied,

BSF instruction skipped, NOP is

executed and then MOLW is executed.

Flags affected: None

BTFSS: Test the corresponding bit in the respective File Register and skip the next instruction if the bit is set. Discard the next instruction that is fetched and execute NOP if the bit is 1, else execute next instruction.

Format & Ex: BTFSS f, b, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x8F, [0x130] = 0x0F

BTFSS 0x30, 3, 0	--> Bit 3 is 1 in 0x8F, So, condition satisfied, BSF instruction skipped, NOP is executed and then MOLW is executed.
BTFSS 0x30, 4, 1	--> Bit 4 is 0 in 0x0F, So, condition not satisfied, BSF and MOLW are executed.

Flags affected: None

Mixed Instructions

TSTFSZ: Test the 8-bit value in the corresponding File Register and skip the next instruction if the value is 0. Discard the next instruction that is fetched and execute NOP if the value is 0, else execute next instruction.

Format & Ex: TSTFSZ f, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x00, [0x130] = 0x8F

Instruction Sequence is

CPFSLT

BSF

MOLW

TSTFSZ 0x30, 0	--> [0x30] is 0x00, So, condition satisfied, BSF instruction skipped, NOP is executed and then MOLW is executed.
TSTFSZ 0x30, 1	--> [0x130] is 0x8F, So, condition not satisfied, BSF and MOLW are executed.

Flags affected: None

CPFSEQ: Compare the 8-bit value in the corresponding File Register with the 8-bit value in W-Register. I.e. perform subtraction File Register – W-Register; skip the next instruction if the value is 0, i.e. equal. Discard the next instruction that is fetched and execute NOP if the value is 0, else execute next instruction. File Register and W-Register content remain unchanged.

Format & Ex: CPFSEQ f, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x20, [0x130] = 0x01

Instruction Sequence is

CPFSEQ

BSF

MOLW

CPFSEQ 0x30, 0	--> [0x30] = 0x20, W-Register = 0x20 So, condition satisfied, BSF instruction skipped, NOP is executed and then MOLW is executed.
CPFSEQ 0x30, 1	--> [0x130] = 0x01, W-Register = 0x20 So, condition not satisfied, BSF and MOLW are executed.

Flags affected: None

CPFSGT: Compare the 8-bit value in the corresponding File Register with the 8-bit value in W-Register. I.e. perform subtraction File Register – W-Register; skip the next instruction if the value is > 0 i.e. $f > w$. Discard the next instruction that is fetched and execute NOP if the value of f is greater, else execute next instruction. File Register and W-Register content remain unchanged.

Format & Ex: CPFSGT f, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x50, [0x130] = 0x01

Instruction Sequence is

CPFSGT

BSF

MOLW

CPFSGT 0x30, 0	--> [0x30] = 0x50, W-Register = 0x20 So, GT condition satisfied, BSF instruction skipped, NOP is executed and then MOLW is executed.
CPFSGT 0x30, 1	--> [0x130] = 0x01, W-Register = 0x20 So, GT condition not satisfied, BSF and MOLW are executed.

Flags affected: None

CPFSLT: Compare the 8-bit value in the corresponding File Register with the 8-bit value in W-Register. I.e. perform subtraction File Register – W-Register; skip the next instruction if the value is < 0 i.e. $f < w$. Discard the next instruction that is fetched and execute NOP if the value is lesser, else execute next instruction. File Register and W-Register content remain unchanged.

Format & Ex: CPFSLT f, a

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x01, [0x130] = 0x50

Instruction Sequence is

CPFSLT

BSF

MOLW

CPFSLT 0x30, 0	--> [0x30] = 0x01, W-Register = 0x20, So, condition satisfied, BSF instruction skipped, NOP is executed and then MOLW is executed.
CPFSLT 0x30, 1	--> [0x130] = 0x50, W-Register = 0x20 So, condition not satisfied, BSF and MOLW are executed.

Flags affected: None

INCFSZ: Increment the 8-bit value present in File Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice and skip the next instruction that is already fetched if result is 0. i.e. execute NOP if result is 0, else execute next instruction.

Format & Ex: INCFSZ f, d, a See INCF

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x8F, [0x130] = 0xFF

Instruction Sequence is

INCFSZ

BSF

MOLW

INCFSZ 0x30, 0, 0 --> W-Register = 0x90, Not 0	So, condition not satisfied, BSF and MOLW are executed.
INCFSZ 0x30, 1, 0 --> [0x30] = 0x90, Not 0	So, condition not satisfied, BSF and MOLW are executed.
INCFSZ 0x30, 0, 1 --> W-Register = 0x00, Is 0,	So, condition satisfied, BSF instruction skipped, NOP is executed and then MOLW is executed.
INCFSZ 0x30, 1, 1 --> [0x130] = 0x00, Is 0,	So, condition satisfied, BSF instruction skipped, NOP is

executed and then MOLW is executed.

Flags affected: None

INCFSNZ: Increment the 8-bit value present in File Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice and skip the next instruction that is already fetched if result is not 0. i.e. execute NOP if result is not 0, else execute next instruction.

Format & Ex: INCFSNZ f, d, a See INCF

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x8F, [0x130] = 0xFF

Instruction Sequence is

INCFSNZ

BSF

MOLW

INCFSNZ 0x30, 0, 0	--> W-Register = 0x90, Not 0 So, condition satisfied, BSF instruction skipped, NOP is executed and then MOLW is executed.
INCFSNZ 0x30, 1, 0	--> [0x30] = 0x90, Not 0 So, condition satisfied, BSF instruction skipped, NOP is executed and then MOLW is executed.
INCFSNZ 0x30, 0, 1	--> W-Register = 0x00, Is 0, So, condition not satisfied, BSF and MOLW are executed.
INCFSNZ 0x30, 1, 1	--> [0x130] = 0x00, Is 0, So, condition not satisfied, BSF and MOLW are executed.

Flags affected: None

DECFSZ: Decrement the 8-bit value present in File Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice and skip the next instruction that is already fetched if result is 0. i.e. execute NOP if result is 0, else execute next instruction.

Format & Ex: DECFSZ f, d, a; See DECF

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x8F, [0x130] = 0x01

Instruction Sequence is

DECFSZ

BSF

MOLW

DECFSZ 0x30, 0, 0--> W-Register = 0x8E, Not 0

So, condition not satisfied,

BSF and MOLW are executed.

DECFSZ 0x30, 1, 0--> [0x30] = 0x8E, Not 0

So, condition not satisfied,

BSF and MOLW are executed.

DECFSZ 0x30, 0, 1--> W-Register = 0x00, Is 0,

So, condition satisfied,

BSF instruction skipped, NOP is
executed and then MOLW is executed.

DECFSZ 0x30, 1, 1--> [0x130] = 0x00, Is 0,

So, condition satisfied,

BSF instruction skipped, NOP is
executed and then MOLW is executed.

Flags affected: None

DECFSNZ: Decrement the 8-bit value present in File Register and store in W-Register or File Register of default access bank or one pointed by BSR Register based on the choice and skip the next instruction that is already fetched if result is not 0. i.e. execute NOP if result is not 0, else execute next instruction.

Format & Ex: DECFSNZ f, d, a See DECF

Assuming W-Register = 0x20, BSR = 1, [0x30] = 0x8F, [0x130] = 0x01

Instruction Sequence is

DECFSNZ

BSF

MOLW

DECFSNZ 0x30, 0, 0

--> W-Register = 0x8E, Not 0

So, condition satisfied,

BSF instruction skipped, NOP is
executed and then MOLW is executed.

DECFSNZ 0x30, 1, 0

--> [0x30] = 0x8E, Not 0

So, condition satisfied,

BSF instruction skipped, NOP is
executed and then MOLW is executed.

DECFSNZ 0x30, 0, 1

--> W-Register = 0x00, Is 0,

DECFSNZ 0x30, 1, 1

So, condition not satisfied,
BSF and MOLW are executed.

--> [0x130] = 0x00, Is 0,

So, condition not satisfied,
BSF and MOLW are executed.

Flags affected: None

Branch & Control Instructions

BC: Branch if Carry - Branch to a location by provided 8-bit relative offset (+127 to -128) from the current location if Carry Flag is set. I.e. $PC = PC + 2 + 2n$; Where $2n =$ up to +127 or -128.

Format & Ex: BC n

Assuming Instruction Sequence is

	MOVWF
	BCF
	BC L1
	BSF
L1	MOLW

L1	MOVWF
	BCF
	BC L1
	BSF
	MOLW

Flags affected: None

BNC: Branch if Not Carry - Branch to a location by provided 8-bit relative offset (+127 to -128) from the current location if Carry Flag is clear. I.e. $PC = PC + 2 + 2n$; Where $2n =$ up to +127 or -128.

Format & Ex: BNC n Refer BC Instruction

Flags affected: None

BN: Branch if Negative - Branch to a location by provided 8-bit relative offset (+127 to -128) from the current location if Negative Flag is set. I.e. $PC = PC + 2 + 2n$; Where $2n =$ up to +127 or -128.

Format & Ex: BN n Refer BC Instruction

Flags affected: None

BNN: Branch if Not Negative - Branch to a location by provided 8-bit relative offset (+127 to -128) from the current location if Negative Flag is clear. I.e. $PC = PC + 2 + 2n$; Where $2n =$ up to +127 or -128.

Format & Ex: BNN n Refer BC Instruction

Flags affected: None

BOV: Branch if Overflow - Branch to a location by provided 8-bit relative offset (+127 to -128) from the current location if Overflow Flag is set. I.e. $PC = PC + 2 + 2n$; Where $2n =$ up to +127 or -128.

Format & Ex: BOV n Refer BC Instruction

Flags affected: None

BNOV: Branch if Not Overflow - Branch to a location by provided 8-bit relative offset (+127 to -128) from the current location if Overflow Flag is clear. I.e. $PC = PC + 2 + 2n$; Where $2n =$ up to +127 or -128.

Format & Ex: BNOV n Refer BC Instruction

Flags affected: None

BZ: Branch if Zero - Branch to a location by provided 8-bit relative offset (+127 to -128) from the current location if Zero Flag is set. I.e. $PC = PC + 2 + 2n$; Where $2n =$ up to +127 or -128.

Format & Ex: BZ n Refer BC Instruction

Flags affected: None

BNZ: Branch if Not Zero - Branch to a location by provided 8-bit relative offset (+127 to -128) from the current location if Zero Flag is clear. I.e. $PC = PC + 2 + 2n$; Where $2n =$ up to +127 or -128.

Format & Ex: BNZ n Refer BC Instruction

Flags affected: None

BRA: Unconditional Branch - Branch to a location by provided 11-bit relative offset (+1023 to -1024) from the current location. I.e. $PC = PC + 2 + 2n$; Where $2n =$ up to +1023 or -1024.

Format & Ex: BRA n

Flags affected: None

GOTO: Unconditional Branch - Branch to a location by provided 20-bit direct address, jump to anywhere in 2-M byte memory. I.e. $PC = k$; or $k \rightarrow PC_{\langle 20:1 \rangle}$; Where $k =$ up to +1048575.

Format & Ex: GOTO n

Flags affected: None

CALL: Subroutine Call – Call a subroutine/module anywhere in the 2-M byte memory by specifying direct 20-bit address. The Return (next instruction) address after CALL instruction i.e $PC + 4$ is pushed on to Stack and $STKPTR_{\langle 4:0 \rangle}$ is incremented by 1. If context save is

enabled by setting $s = 1$, the W, Status and BSR register values are stored into their respective shadow registers and PC is loaded with the 20-bit direct address. $PC = k$; or $k \rightarrow PC<20:1>$; Where $k =$ up to +1048575.

Format & Ex: CALL n, s

Assuming W-Register = 0x30, BSR = 3, STATUS = 0xFF

CALL L1, 0 --> The Return address (PC + 4) copied to stack
 STKPTR<4:0> is incremented by 1
 PC loaded with direct address of L1

CALL L1, 1 --> The Return address (PC + 4) copied to stack
 STKPTR<4:0> is incremented by 1
 W, BSR, Status Register copied to Shadow Reg
 PC loaded with direct address of L1

Flags affected: None

RCALL: Relative Subroutine Call – Call a subroutine/module, jump up to 1K from current location using relative 11-bit address. The Return (next instruction) address after RCALL instruction i.e PC+ 2 is pushed on to Stack and STKPTR<4:0> is incremented by 1. PC is added with the 2's complement 11-bit direct address. $PC = PC + 2 + 2n$; Where $2n =$ upto +1023 or -1024.

Format & Ex: RCALL n

Flags affected: None

Assuming

RCALL L1 --> The Return address (PC + 2) copied to stack
 PC loaded with L1 - 2's complement $2n$ relative add
 $PC = PC + 2 + 2n$

RETURN: Return from Subroutine Call – Return from a subroutine/module anywhere in the 2-M byte memory, direct 20-bit address popped from Stack through TOS register and STKPTR<4:0> is decremented by 1. If context save is enabled by setting $s = 1$, the W, Status and BSR register values are restored/copied back from their respective shadow registers $PC = TOS$;

Format & Ex: RETURN s

Assuming W-Register = 0x30, BSR = 3, STATUS = 0xFF

RETURN 0 --> PC loaded with direct return address.
 The Return address (PC + 4) copied from stack
 TOS; STKPTR<4:0> decremented by 1

RETURN 1 --> PC loaded with direct return address

The Return address (PC + 4) copied from stack
TOS; STKPTR<4:0> decremented by 1
W, BSR, Status Register restore/copied from
Respective Shadow Registers

Flags affected: None

RETLW: Return Literal to W-Register. The 8-bit literal value is copied to W-Register and the Program Counter – PC is loaded from Stack – TOS (Return Address). K --> W-Register; (TOS) --> PC; PCLATH & PCLATU are unchanged.

Format & Ex: RETLW k

RETLW 0x23 --> W-Register = 0x23, PC = [TOS]

Flags affected: None

RETFIE: Return from Interrupt – Return back from Interrupt Service Routine. The stack is popped and TOS contents are copied to PC, STKPTR<4:0> decremented by 1, Interrupts are enabled – GIE/GIEH or PEIE/GIEL = 1. If context save is enabled by setting s = 1, the W, Status and BSR register values are restored/copied back from their respective shadow registers. PC = TOS;

Format & Ex: RETFIE s

RETFIE 0 --> POP to PC, PC = [TOS]; STKPTR<4:0> decremented by 1

Enable GIE/GIEH or PEIE/GIEL = 1

RETFIE 1 --> POP to PC, PC = [TOS]; STKPTR<4:0> decremented by 1

Enable GIE/GIEH or PEIE/GIEL = 1

W, BSR, Status Register restore/copied from
Respective Shadow Registers

Flags affected: GIE/GIEH, PEIE/GIEL

RESET: Reset – Software instruction to reset all registers and Flags affected by a MCLR Reset.

Format & Ex: RESET

RESET --> Reset all Registers & Flags

Flags affected: All

CLRWDT: Clear Watchdog Timer – Resets the Watchdog Timer and respective prescaler. PD – Power Down status bit is set to 1 and TO – Time-Out status bit is set to 1.

Format & Ex: CLRWDT

CLRWDT

Flags affected: TO, PD

SLEEP: Enter Sleep Mode – The PD – Power Down status bit is cleared and TO – Time-Out status bit is set. The postscaler and Watchdog Timer are also cleared, and device is put into Sleep mode by stopping the oscillator.

Format & Ex: SLEEP --> WDT = 0x00, WDT prescaler =0, PD = 0, TO = 1.

Flags affected: TO, PD

4.6 Overview of MPLAB Simulator

The MPLAB simulator may be downloaded from the following Microchips Webpage.

<https://www.microchip.com/en-us/tools-resources/archives/mplab-ecosystem>

Under the MPLAB X IDE archives, you may select MPLAB X v5.00, MPLAB X v5.05, MPLAB X v5.10, MPLAB X v5.15 or MPLAB X v5.20. The following documentation is based on MPLAB X v5.10 version. Other variations may be used but there could be change in windows or the available options.

Simulating PIC 18 C programs makes use of XC8 compiler. This may also be downloaded from the following website.

<https://www.microchip.com/en-us/tools-resources/develop/mplab-xc-compilers>

The procedure for simulating Assembly and C Language programs using MPLAB simulator is presented here.

1. Install the MPLAB X IDE v5.10 from the weblink provided above.
2. After the installation, select and click on the shortcut of MPLAB X IDE v5.10 in the start window. The following window opens as shown in Fig 4.7.
3. In the File menu select the “New Project” option. The following window opens as shown in Fig 4.8.

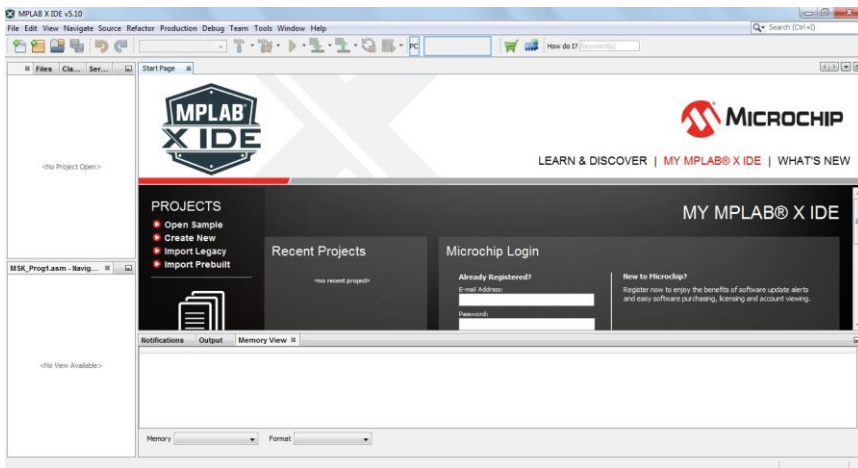


Fig 4.7 MPLAB window

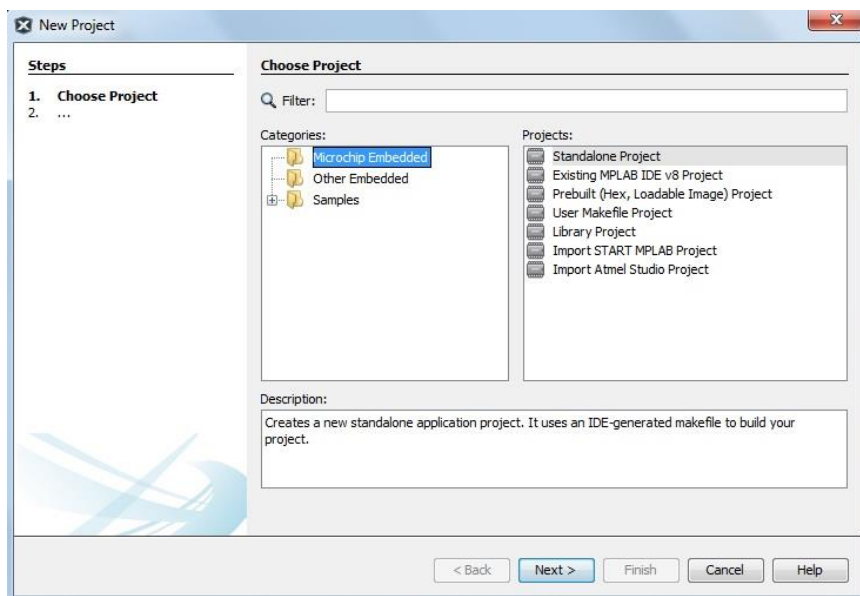


Fig 4.8 New Project Window

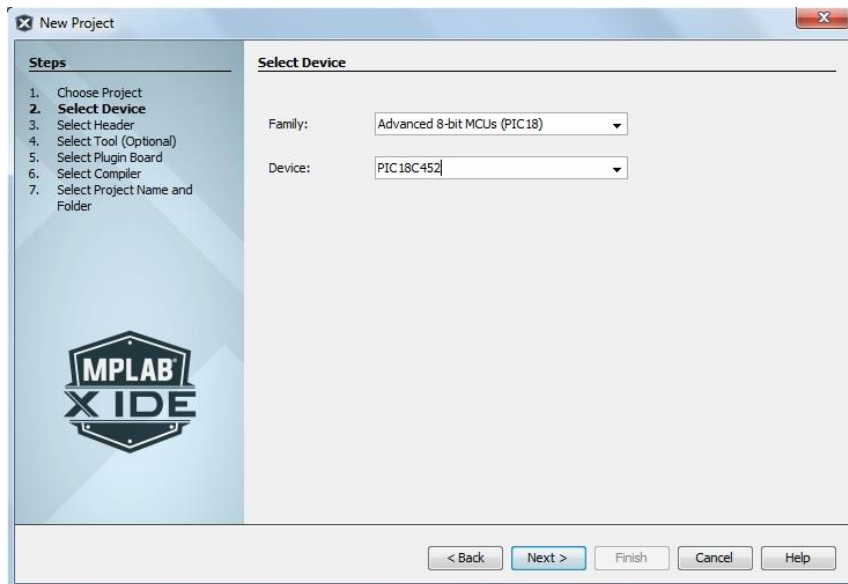


Fig 4.9 New Project – Select Device Window

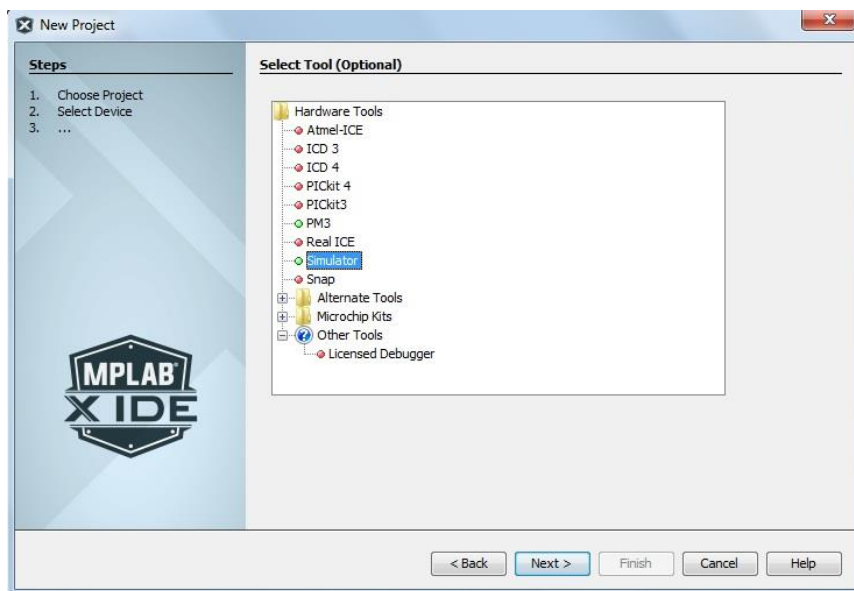


Fig 4.10 New Project – Select Tool (Optional) Window

4. In the “New Project” window under Categories select “Microchip Embedded” and then under Projects select “Standalone Project” option and click Next. The following window appears as shown in Fig 4.9.

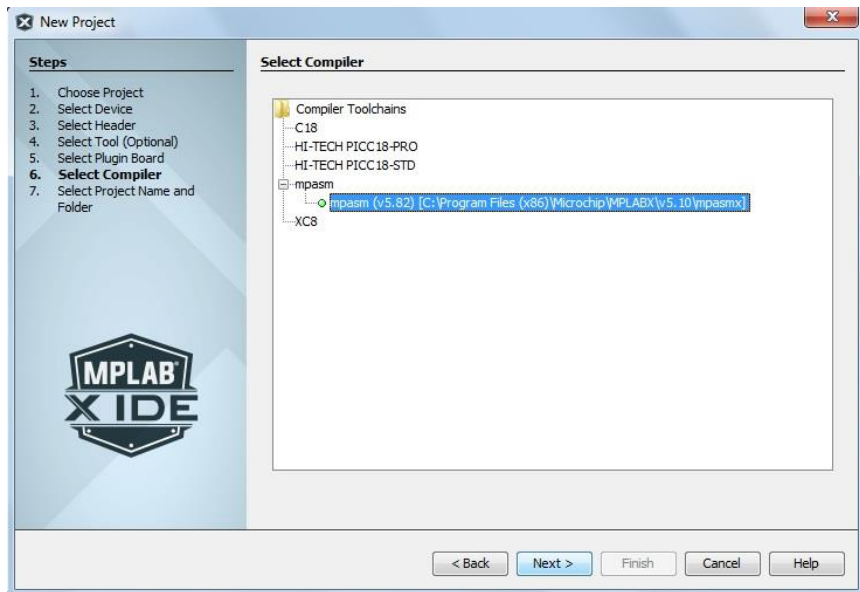


Fig 4.11 New Project – Select Compiler Window

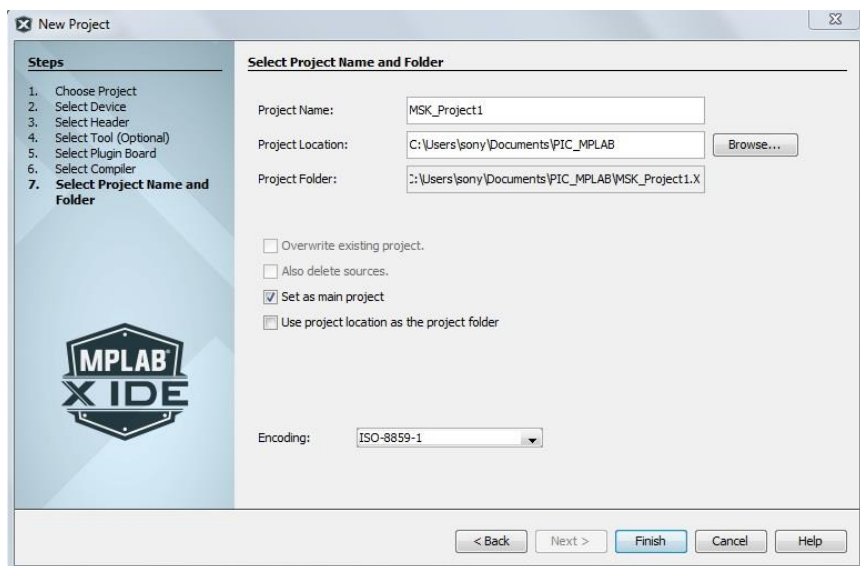


Fig 4.12 New Project – Select Project Name and Folder Window

5. In the “Select Device” window in the Family section select “Advanced 8-bit MCUs (PIC18)” option. Then in the Device section select PIC18F452 or PIC18F458 or PIC18C452/458 and then click Next. A Select Tool option window appears as shown in Fig 4.10.

6. In the “Select Tool” window under Hardware Tools section select “Simulator” option. Click Next and a “Select Compiler” window appears as shown in Fig 4.11.
7. For simulating assembly language programs, under “mpasm” section select “mpasm (v5.82)” and click Next. A “Select Project Name & Folder” window appears as shown in Fig 4.12.
8. For simulating C program an XC8 compiler is required This may also be downloaded from the following website and that respective C compiler like “Hi-TECH PICC 18-STD” may be selected from the list of options.
9. <https://www.microchip.com/en-us/tools-resources/develop/mplab-xc-compilers>
10. Enter the Project Name and select the desired project location and click on Next. Your MPLAB project is created, and window appears as shown in Fig 4.13a.

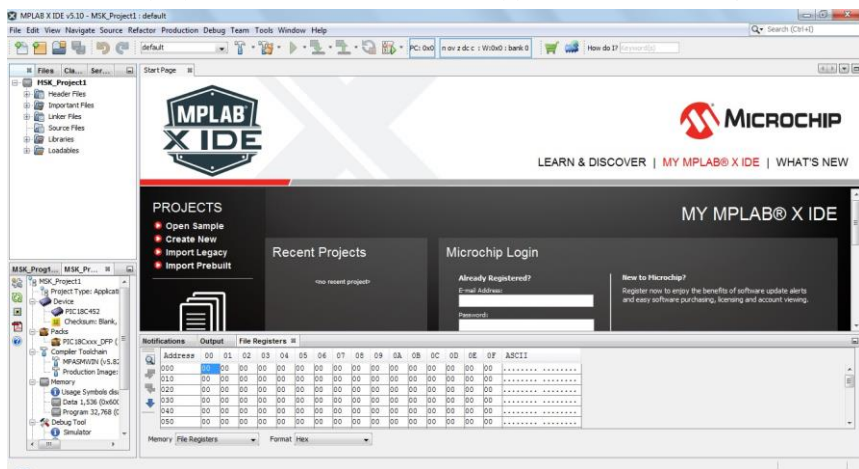


Fig 4.13a Your Project Window “MSK_Project1”

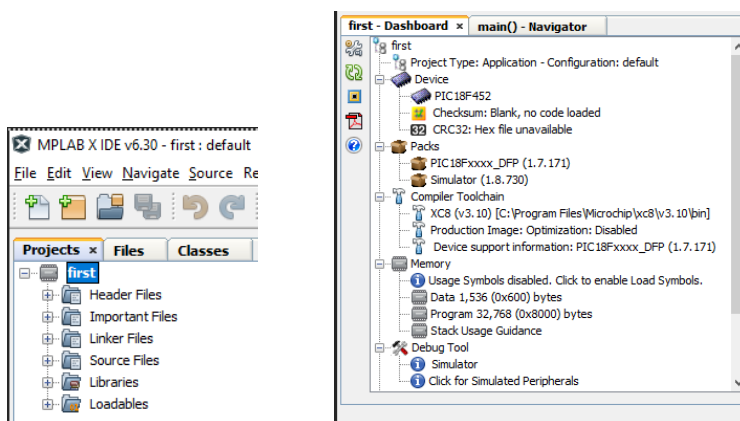


Fig 4.13b Your “first” Project Window

Fig 4.13c Dashboard

The projects “first” window is shown in Fig 4.13b where you can see the

files related to the project and the dashboard window is shown in Fig 4.13c. On the left vertical tab, you can see some icon options to see project properties, update selected debug tool, alternate enable/disable breakpoint, web-based version of datasheet in pdf, help file for compiler used etc. Under the “first” dashboard you could see selected MCU, Program Checksum, build tool - Compiler tool chain XC8 with version 3.1, Packs, Memory usage, Debug tools etc.

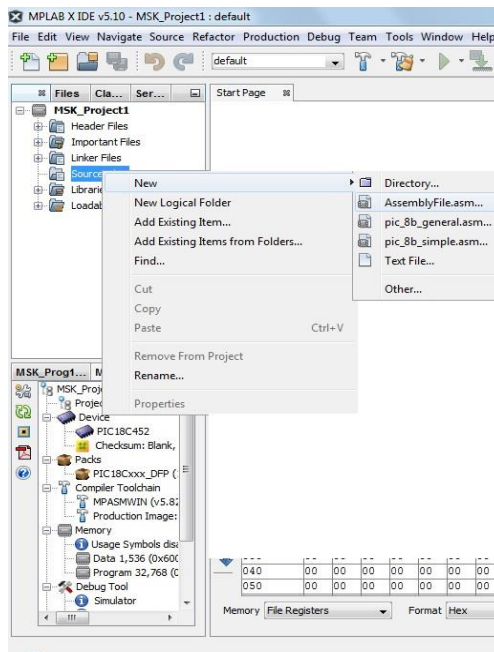


Fig 4.14 Your Project Window

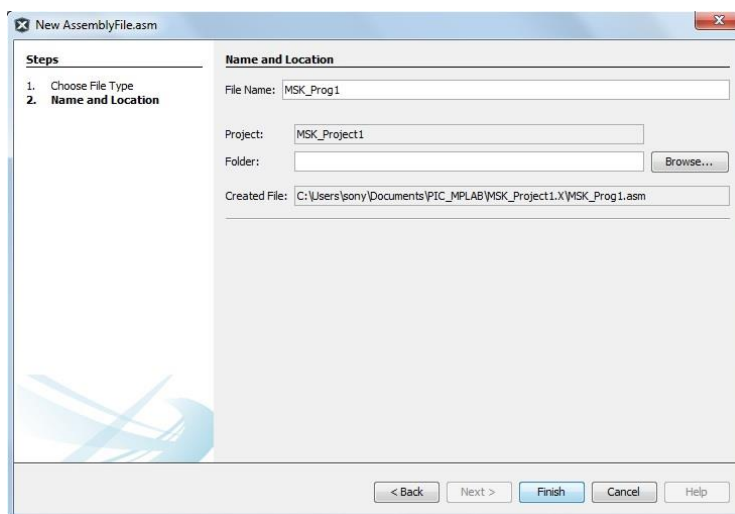


Fig 4.15 New Assembly File.asm Window

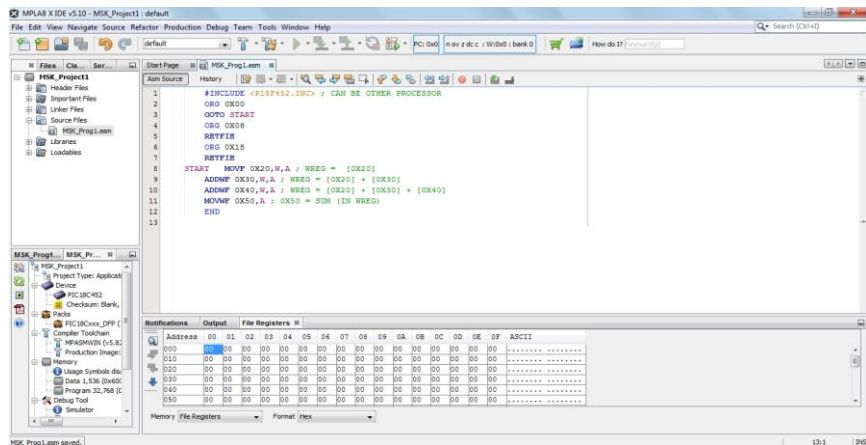


Fig 4.16 Editor Window for ASM file

11. On the Top Left side under your project section right click on “Source Files”. Click on “New” and select “AssemblyFile.asm” option as shown in Fig 4.14.
12. A “New Assembly File.asm” Window appears as shown in Fig 4.15. Enter the name of the file and select the location to save the file and click Next.
13. The ASM Editor window appears as shown in Fig 4.16. Enter you assembly language program to simulate and save it.
14. To change project properties, you may right click on your project and select “Properties” option, and you may change the family, device, simulator etc as shown in Fig 4.17.

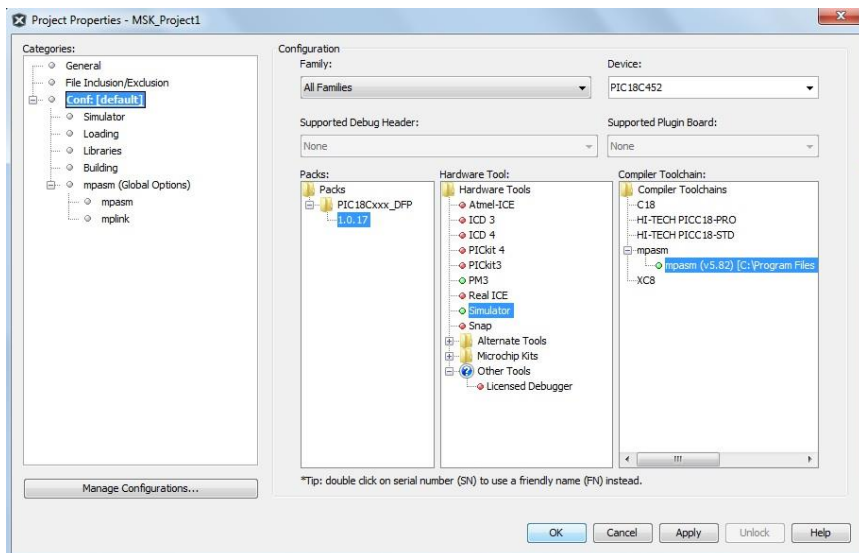


Fig 4.17 Project Properties Window

15. The project can be built by right clicking on your project and selecting “Build” option as shown in Fig 4.18.

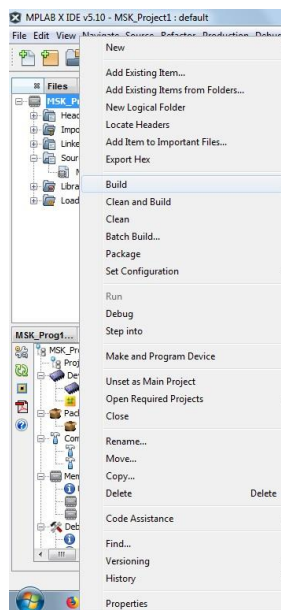


Fig 4.18a Build Project



Fig 4.18b Build Icon

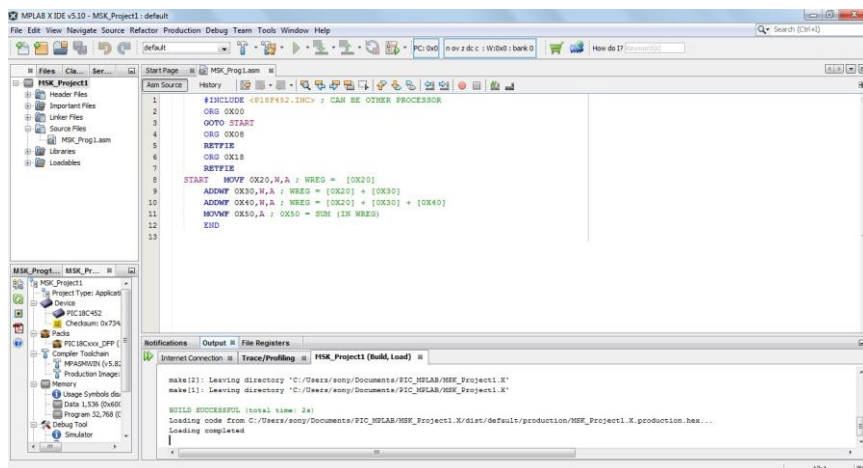


Fig 4.19 Output Window - Successful Build Project

16. When the build is successful you see Build Successful in Output window as shown in Fig 4.19.
17. To debug, click debug icon or “Debug” menu & select “Debug Main Project” as shown in Fig 4.20. The debugger console will run as shown in Fig 4.20c.

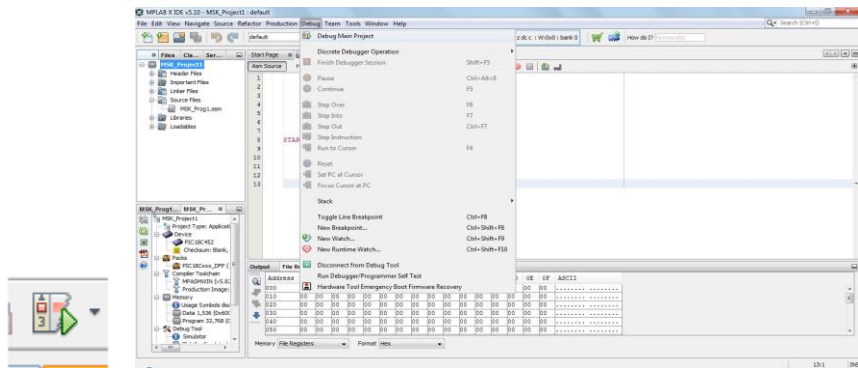


Fig 4.20 a) Debug Icon

Fig 4.20 b) Debug

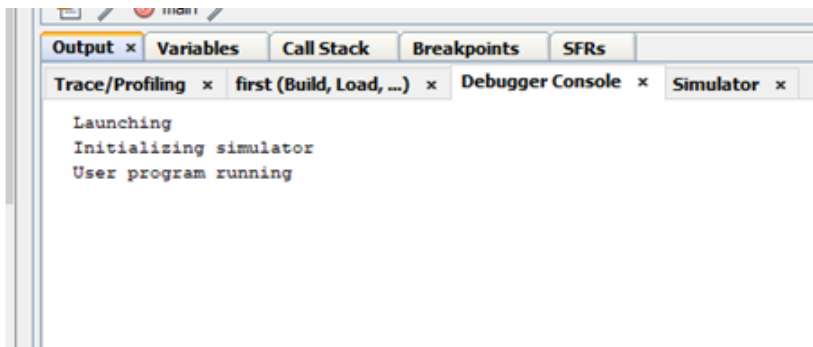


Fig 4.20c Debugger Console

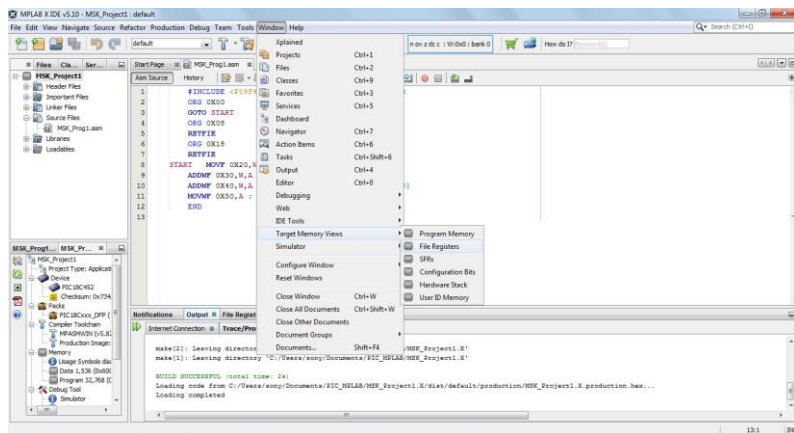


Fig 4.21 Target Memory Views

18. Under the “Window” menu select “Target Memory Views” (Fig 4.21) to see the “Program Memory”, “File Registers”, “SFRs”, “Configuration Bits”. Select File Registers to see the corresponding RAMs File Register memory locations or SFRs to see the special function registers like Status register, FSR, peripheral interrupt registers, etc. It is as shown in Fig 4.21.

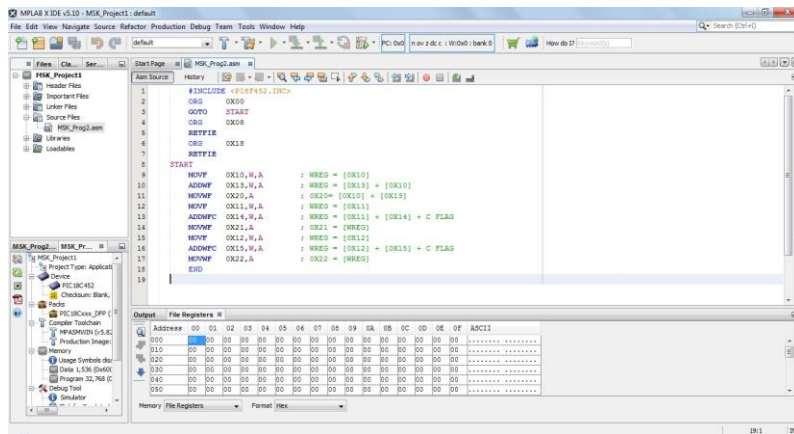


Fig 4.22a Value Entry to Memory

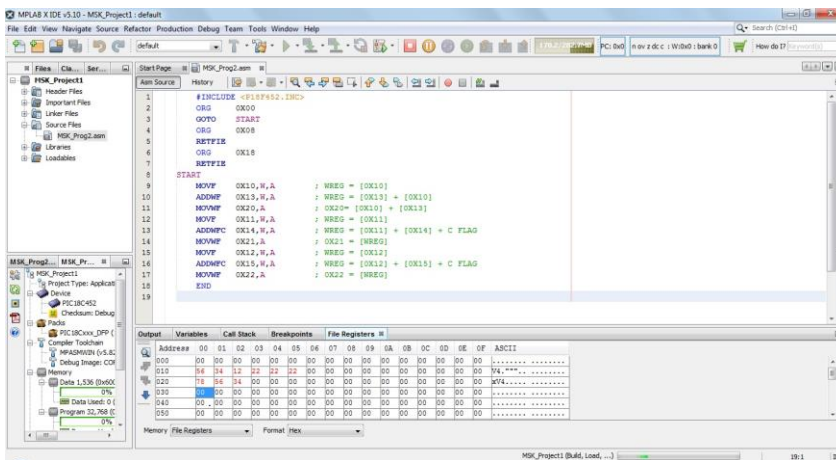


Fig 4.22b Value Entered Memory

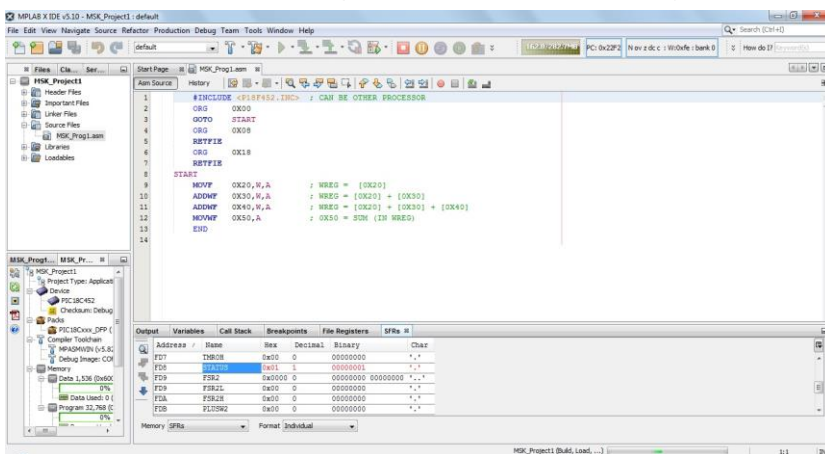


Fig 4.23 SFRs

19. You may enter the required values in the respective File Register - memory locations as shown in Fig 4.22 by double clicking on respective location. The SFR window is shown in Fig 4.23.

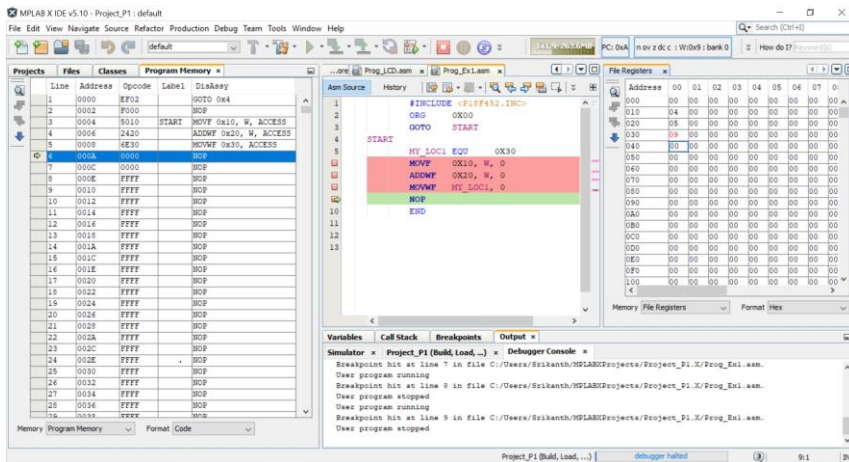


Fig 4.24 Breakpoints

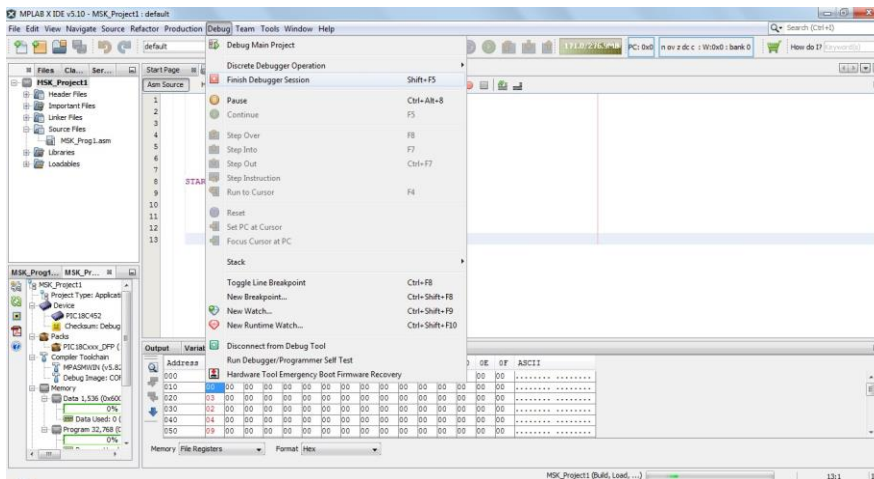


Fig 4.25 Finish Debugger Session

20. Breakpoints may be added to see step wise execution of the program by clicking on the line numbers shown on the left side of the editor as shown in Fig 4.24. The execution pauses or stops the execution of instructions at every breakpoint and waits for the user to continue (highlighted in green). User may press F5 or click green arrow to proceed.

Users could see Pink square boxes at the beginning of the lines on the left side and have instruction lines highlighted in pink wherever break points are inserted. The green highlight indicates the PC PTR or step or the instruction that is going to be executed next.

21. To finish debug, click on “Debug” menu and select “Finish Debugger Session” as shown in Fig 4.25.

For C Programming

22. Create a “main.c” as shown in Fig 4.26 by right clicking on “source files” --> “New” --> “main.c” in your “first” projects window.

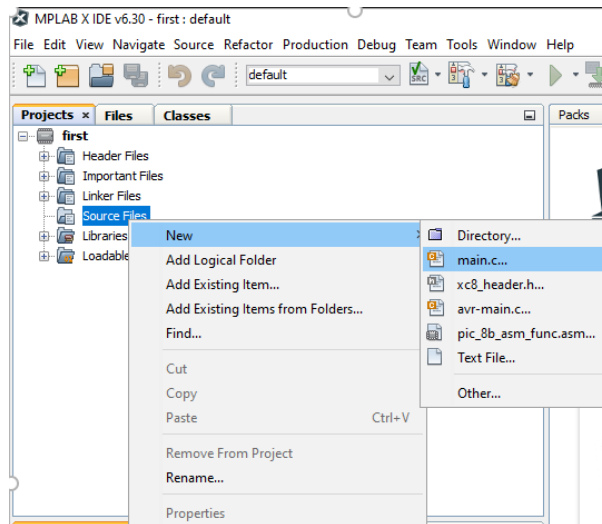


Fig 4.26 C program – main.c creation

23. A new main.c file steps window appears as shown in Fig 4.27 where you assign a file name, path etc.
24. Enter the required file name, select location and click Finish. An editor window appears where you can enter the required C-Program as shown in Fig 4.28.

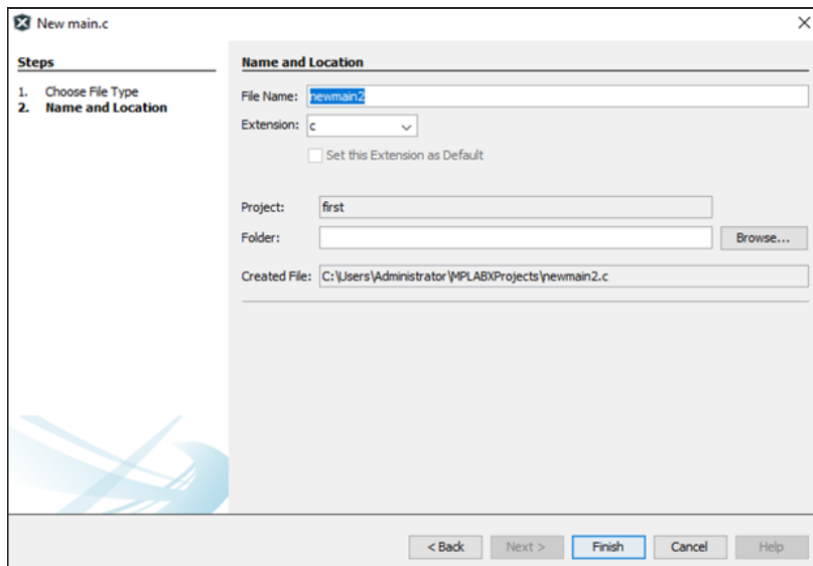


Fig 4.27 New main.c file steps window

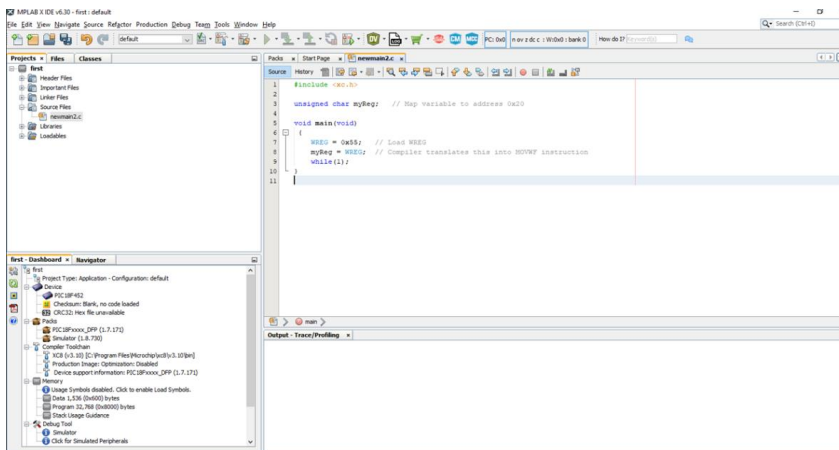


Fig 4.28 Editor - main.c window

A sample C program is presented here.

```
#include <xc.h>

unsigned char myReg;    // Map variable to address 0x20

void main(void)
{
    WREG = 0x55;        // Load WREG
    myReg = WREG;        // Compiler translates this into MOVWF instruction
    while(1);
}
```

25. Click on the icon as shown in Fig 4.29 which lies in the menu or as shown in Fig 4.18 presented in step 15 to build the project.



Fig 4.29 Build Icon

26. Watch for the build success in the output window as shown in Fig 4.30.

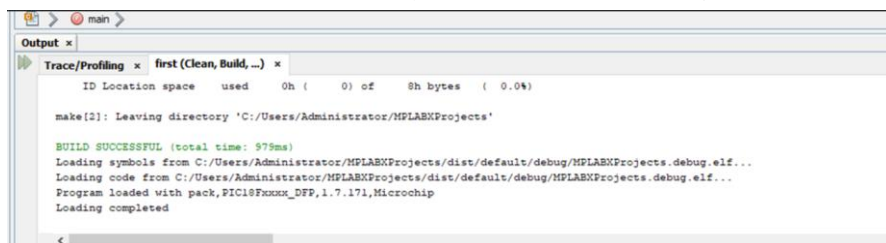


Fig 4.30 Output Window

27. Click on the icon as shown in Fig 4.20 which lies in the menu or as shown in Fig 4.31 presented in step 17 to build the project.



Fig 4.31a) Debug Icon

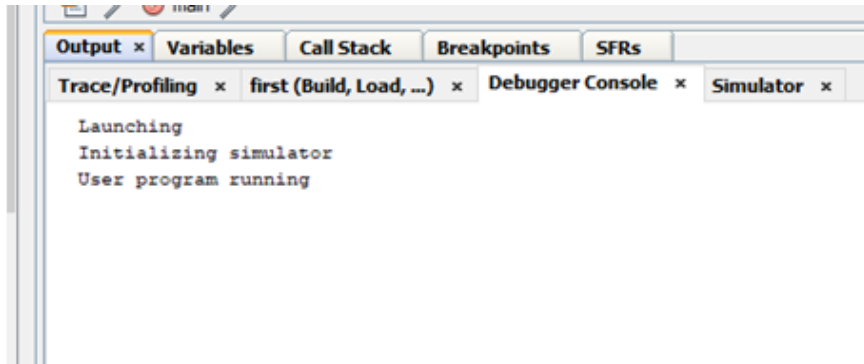


Fig 4.31b) Debugger Console

28. To the SFR registers and content, select Windows --> Target Memory --> SFR's as shown in Fig 4.32.

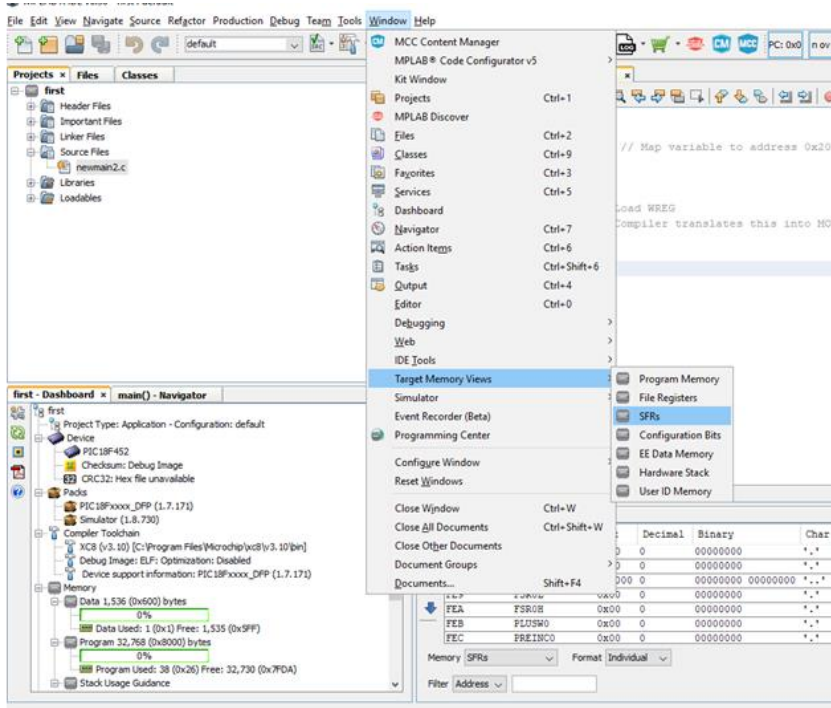


Fig 4.32 SFR Window Selection

29. Watch the literal value of WREG in the SFR window as shown in Fig 4.33.

FE4	PREINC1	0x00	0	00000000	'.'
FE5	POSTDEC1	0x00	0	00000000	'.'
FE6	POSTINC1	0x00	0	00000000	'.'
FE7	INDF1	0x00	0	00000000	'.'
FE8	WREG	0x55	85	01010101	'U'
FE9	FSR0	0x0000	0	00000000 00000000	'..'
FE9	FSR0L	0x00	0	00000000	'.'

Fig 4.33 SFR Window